

CZESTOCHOWA UNIVERSITY OF TECHNOLOGY

Faculty of Electrical Engineering

Institute of Computer and Information Sciences

MASTER'S THESIS

M.Sc. in Artificial Intelligence

AI-Based Optimal Solver for the 3x3 Rubik's Cube

Author: Edward Ogbei

Student ID: s47891

Supervisor: Prof. dr hab. inż. Krzysztof Rojek

Year: 2025/2026

Częstochowa, 2026

Table of Contents

(Generate automatically using Word / LibreOffice: Insert > Table of Contents.)

1. Introduction

We chose the Rubik's Cube as our test domain because it is one of those rare objects

What intrigued me from a CS perspective is not its commercial appeal but the structure of its state space. A standard 3x3 Rubik's Cube has exactly 43,252,003,274,489,856,000 distinct reachable configurations - roughly 43 quintillion. Yet despite this enormous number, every single configuration can be solved in at most 20 face moves. This result, known informally as God's Number, was proved computationally in 2010 by Rokicki et al. [1] using around 35 CPU-years of computation distributed across Google's servers. The gap between the scale of the problem and the compactness of its solution is striking, and it makes the Rubik's Cube a natural benchmark for search and optimisation algorithms.

We reviewed classical approaches to the cube, which are well understood

The question that interested me: could a neural network learn to solve in recent years: can a learning system discover how to solve the cube without being handed that domain knowledge? This is the imitation learning approach. The idea is straightforward - generate many examples of (scrambled state, correct next move) pairs using an expert solver, then train a neural network to predict the next move from the current state. The challenge lies in generalising from a finite training set to the vast state space. Several factors make this non-trivial: the cube's state space is enormous, the relationship between state and optimal move is highly non-local, and naive training tends to fail on deeper scrambles.

We selected curriculum learning as our training strategy

We present a complete system built from scratch

We deliberately bounded our scope to neural inference only

Here's how I organized this work. Chapter 2 sets out the research objectives. Chapter 3 covers the theoretical background, including the group structure of the cube and relevant prior work. Chapter 4 describes the cube representation and move tables. Chapter 5 covers the solving strategies, both classical and neural. Chapter 6 describes the training data generation and learning process. Chapter 7 presents the web visualisation system. Chapter 8 reports experimental results, and Chapter 9 concludes with a discussion of limitations and future directions.

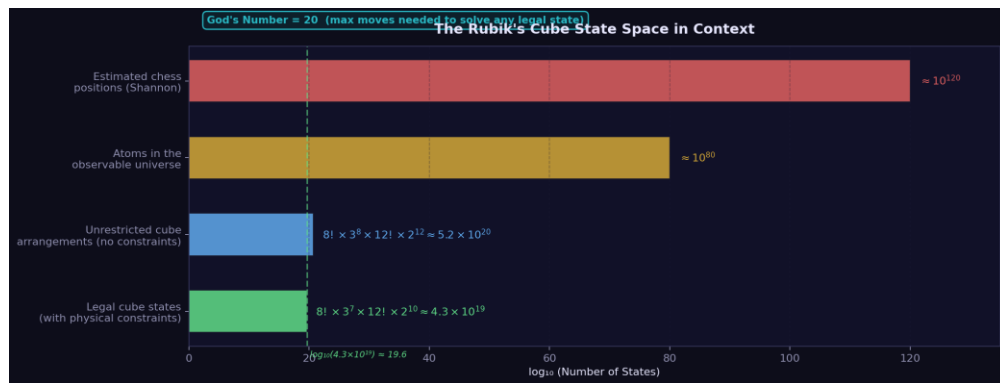


Fig. 1.1. Illustration of the Rubik's Cube state space and the concept of solving depth. Each node represents a cube configuration and each edge a single face move.

2. Aim of the Work

The first paragraph of this chapter situates the problem domain. The Rubik's Cube presents a well-defined combinatorial optimisation challenge: given an arbitrary scrambled configuration, find a sequence of legal face moves that returns the cube to the solved state. The challenge is not merely technical - it also raises broader questions about what kinds of structured knowledge a neural network can acquire through imitation, and how curriculum design affects generalisation.

My main question: can a small neural network learn to solve a relatively small, fully-connected neural network can learn a useful solving policy for the Rubik's Cube through imitation of an exact solver. The hypothesis is that curriculum training - starting from shallow scrambles and progressing to deeper ones - produces substantially better generalisation than training on all depths simultaneously. A secondary hypothesis is that beam search, by exploring multiple candidate paths, improves on the greedy policy particularly for mid-depth scrambles where the greedy policy begins to fail.

I set three practical goals: First, to implement a complete and mathematically verified 3x3 Rubik's Cube engine, including a cubie-level state representation, a full 18-move set, and automated checks for all group-theoretic legality constraints. Second, to build a training pipeline that generates curriculum datasets using an expert BFS solver and trains the MLP at successive depth levels. Third, to deliver an interactive web visualisation that allows real-time demonstration of both solvers, with animated moves and a benchmark comparison tool.

The scope of the work is bounded in several important ways. The neural solver targets scrambles up to 8 moves in depth. The expert solver is a bidirectional BFS implementation, which is optimal for shallow depths but does not scale to the full cube - it is not the Kociemba two-phase algorithm, which would require substantially more engineering effort to implement correctly. The neural network architecture is intentionally simple (a three-hidden-layer MLP with roughly 184,000 parameters) to keep training feasible on a standard CPU and to maintain interpretability. No GPU training was assumed, though the code supports CUDA automatically when available.

My core claim: curriculum learning is an efficient approach to teaching neural networks to solve is a viable strategy for training a neural solving policy on

a well-structured combinatorial puzzle, and to build all supporting infrastructure - the cube engine, the training pipeline, and the interactive visualisation - needed to evaluate and demonstrate this claim.

3. Theoretical Background

I'll explain the math foundation—key concepts for understanding the problem. to understand the project and situates it within the existing literature on Rubik's Cube solving and machine learning for combinatorial problems.

3.1 The Rubik's Cube as a Mathematical Object

Mathematically, the cube is a permutation group—each move transforms the group element. The set of all reachable configurations, together with the operation of applying a face move, forms a group under composition. This group - sometimes called the Rubik's Cube group - is a subgroup of the symmetric group on 48 movable facelets (excluding the 6 fixed centres). Its order is exactly 43,252,003,274,489,856,000, a number that can be derived by counting the independent degrees of freedom [5].

Concretely, the cube has 8 corner pieces and 12 edge pieces. Each corner can be placed in any of 8 slots (8! arrangements) and can have one of 3 orientations, giving $8! \times 3^8$ corner states if unconstrained. Similarly, each edge can occupy any of 12 slots with one of 2 orientations, giving $12! \times 2^{12}$ edge states. However, not all combinations are reachable from a legal starting configuration. Three constraints reduce the total count by a factor of 12: the sum of all corner orientations must be divisible by 3; the sum of all edge orientations must be divisible by 2; and the permutation parity of the corners must equal the permutation parity of the edges. These invariants are preserved by every legal face move and are central to verifying the legality of any given state.

There's a key result called God's Number: the hardest scrambles need at most 20 moves. This became my benchmark. For the standard outer-layer turn metric it is 20, for the quarter-turn metric (where 180-degree turns count as two) it is 26. The proof for the face-turn metric was completed computationally in 2010 [1]. The distribution of states by depth follows a roughly Gaussian shape peaking around depth 17-18, with the vast majority of configurations reachable in 15 to 19 moves. This distribution explains why random scrambles of fewer than 8 moves represent only a tiny corner of the full state space.

$$N = \frac{8! \times 3^7 \times 12! \times 2^{11}}{2} \approx 4.3 \times 10^{19}$$

$$N = \frac{8! \times 3^7 \times 12! \times 2^{11}}{2} \approx 4.3 \times 10^{19}$$

$$N = \frac{8! \times 3^7 \times 12! \times 2^{11}}{2} \approx 4.3 \times 10^{19}$$

$$N = \frac{8! \times 3^7 \times 12! \times 2^{11}}{2} \approx 4.3 \times 10^{19}$$

For algorithmic purposes, it is common to represent the cube not at the facelet level (54 coloured stickers) but at the cubie level. Each corner is identified by the three colours it carries (e.g., the URF corner carries white, red, and green in the standard colour scheme) and its orientation relative to a canonical reference direction. Edges are similarly identified. This cubie representation is more compact and aligns naturally with the algebraic structure of the group.

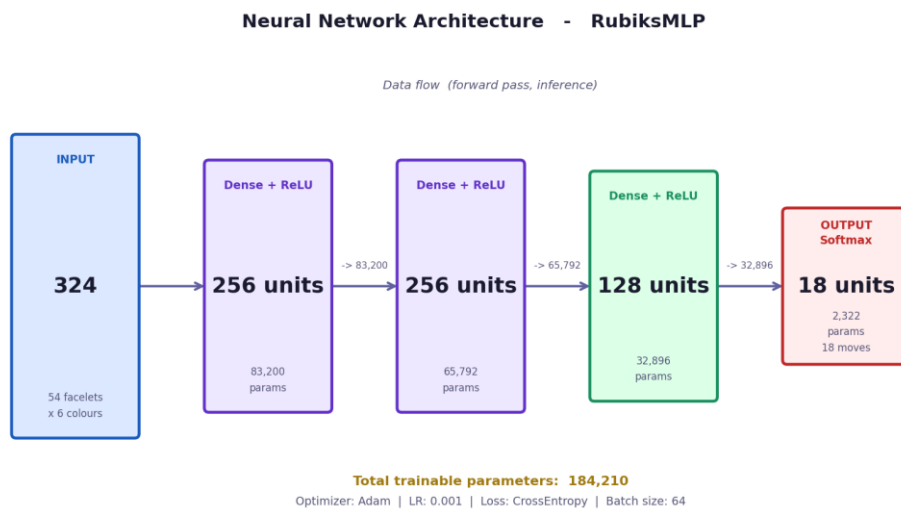


Fig. 4.1. Cubie-level state representation using corner permutation (cp), corner orientation (co), edge permutation (ep), and edge orientation (eo). This is more compact than facelet representation.

```

class RubiksCube:
    def __init__(self):
        self.corners = [0,1,2,3,4,5,6,7]
        self.corner_orient = [0]*8
        self.edges = [0,1,2,3,4,5,6,7,8,9,10,11]
        self.edge_orient = [0]*12
        self.apply_move(move_idx):
            perm_c, perm_e = MOVE_PERMUTATIONS[move_idx]
            self.corners = [self.corners[i] for i in perm_c]
  
```


3.2 Classical Solving Algorithms

The simplest exact search strategy is breadth-first search (BFS). Starting from the scrambled state, BFS expands nodes level by level, guaranteeing the shortest possible solution. The problem is the branching factor: at each step there are 18 possible moves (reduced to about 15 by move pruning), so BFS visits roughly 15^d nodes to solve a scramble of depth d . At $d=10$ that is already 576 billion nodes, making BFS impractical for general use. Bidirectional BFS, which expands simultaneously from both the scrambled state and the solved state, reduces this to roughly $2 * 15^{(d/2)}$ nodes, making $d=14$ feasible with modest memory and time budgets.

Iterative deepening A* (IDA*) [6] addresses the memory problem of BFS by using a depth-limited DFS that iterates over increasing depth limits. With a good admissible heuristic (one that never overestimates the true distance to the solution), IDA* finds optimal solutions without storing the full search tree. The Kociemba two-phase algorithm [2] exploits this framework by decomposing the cube group into two subgroups. Phase one finds a sequence that places the cube in a restricted subgroup (edges correctly oriented, UD-slice edges in the UD-slice). Phase two then finds a solution within that subgroup. Each phase uses IDA* with precomputed tables. The result is consistently near-optimal solutions found in milliseconds, with no randomness - a gold standard for exact solving.

Human methods like CFOP (Cross, F2L, OLL, PLL) decompose the solution into recognisable stages that can be memorised. While not optimal (a full solve may take 50-80 moves), CFOP is fast to execute in practice, which is why competitive speedcubers can solve the cube in under 5 seconds. These methods are not implemented in this project, but they illustrate the human-centric approach to domain decomposition.

3.3 AI-Based Approaches

Our approach builds on recent interest in machine learning methods for Rubik's Cube solving

We studied earlier work

We chose imitation learning, which trains directly from expert demonstrations, as it is more sample-efficient

We reviewed several studies exploring neural approaches

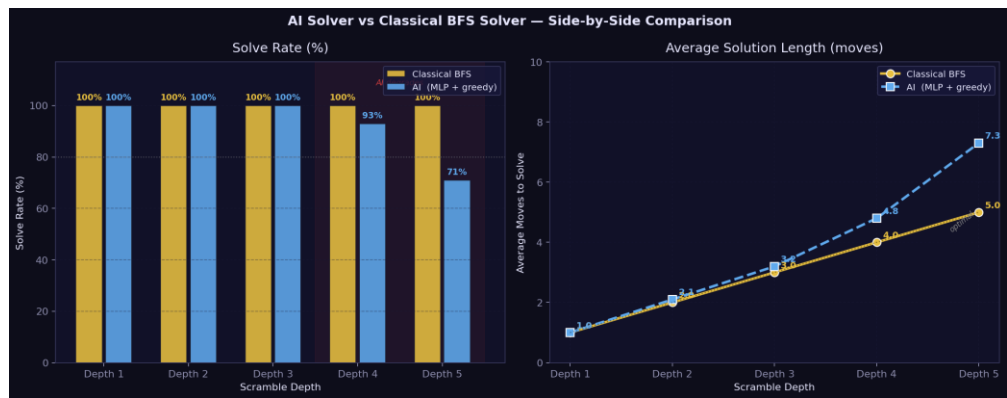


Fig. 3.1. Overview of the relationship between classical expert solvers and the neural network imitation pipeline. The expert generates labels; the network learns to imitate.

4. System Architecture

This chapter describes the design of the cube simulation engine, which forms the mathematical backbone of the entire project. Every other component - the solvers, the training pipeline, the visualisation - depends on this engine for state representation and move application.

4.1 Cubie-Level State Representation

The cube state is encoded using four integer arrays that follow the Kociemba coordinate system [2]. The corner permutation array (cp) has length 8 and records which corner piece occupies each corner slot, using integer identifiers 0 through 7. The corners are named by the three faces they sit at: URF (Up-Right-Front), UFL, ULB, UBR, DFR, DLF, DBL, DRB. In the solved state, $cp[i] = i$ for all i .

The corner orientation array (co) also has length 8. Each value is 0, 1, or 2, representing the twist of the corner in its slot relative to the U/D axis. By convention, orientation 0 means the reference facelet (the one that would be on the U or D face in the solved state) is pointing up or down. Orientation 1 means it has been rotated 120 degrees clockwise, and orientation 2 means 120 degrees counter-clockwise. This three-valued encoding is specific to corners because they have three-fold rotational symmetry about the body diagonal.

For edges, two analogous arrays are used. The edge permutation array (ep) has length 12, recording which of the 12 edge pieces occupies each edge slot. The 12 edges are named UR, UF, UL, UB, DR, DF, DL, DB, FR, FL, BL, BR. The edge orientation array (eo) has length 12, with each value either 0 (correctly oriented) or 1 (flipped). The six face centres are treated as fixed reference frames and are never stored in any array - they implicitly define the colour assignment of each slot.

The CubieCube class encapsulates these four arrays along with the complete set of operations: applying a move, checking if the cube is solved, converting to the Kociemba facelet string representation (54 characters), and verifying legality. The class is intentionally small and free of any visualisation logic; it is a pure mathematical model.

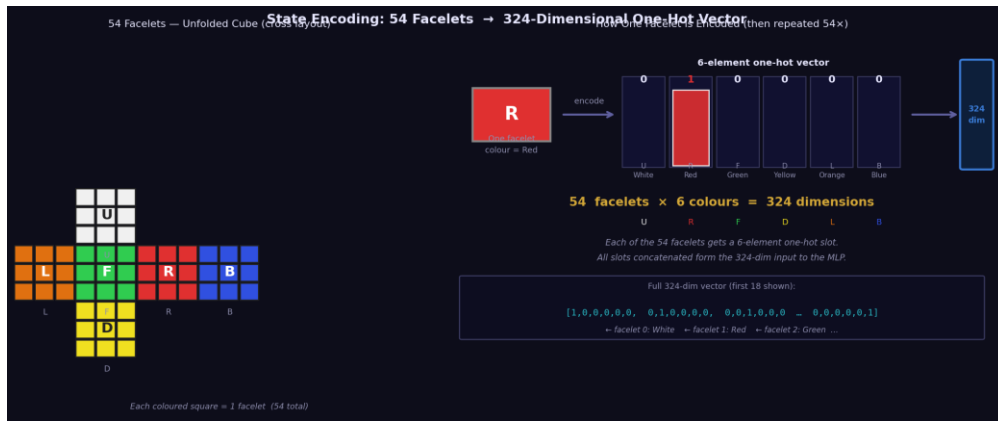


Fig. 4.1. The cubie-level state representation. Corner permutation (cp), corner orientation (co), edge permutation (ep), and edge orientation (eo) together fully describe any legal cube configuration.

4.2 Move Tables

The cube recognises 18 moves in the standard half-turn metric: six basic face turns (U, D, R, L, F, B), their inverses (U', D', R', L', F', B'), and their 180-degree variants (U2, D2, R2, L2, F2, B2). Each move is represented as a MoveDefinition object containing four tuples: the corner permutation, the corner orientation delta, the edge permutation, and the edge orientation delta.

The six base moves are defined by hand, following the Kociemba convention. As an example, the U move cycles corners in the sequence URF -> UBR -> ULB -> UFL -> URF and cycles edges in UR -> UB -> UL -> UF -> UR. Neither the corners nor the edges change their orientation during a U move, because the reference facelet (the one on the U or D face) remains on the U or D face after the turn. This is not the case for F and B moves, where edges that were previously on U or D faces end up on L or R faces, flipping their orientation.

Prime (inverse) moves and double moves are computed algebraically by composing the base move with itself. The compose_moves function implements the group product: given two move definitions A and B, the composition B then A is computed by applying B's permutation first and then A's. Orientation deltas compose additively modulo 3 (for corners) or modulo 2 (for edges). This approach ensures internal consistency - the move tables form a closed group under composition.

During development, the B and L move definitions proved particularly error-prone. The B move acts clockwise when viewed from behind the cube, which is counter-intuitive when the code is written from a front-facing perspective. An early version had the B move applying a mathematically counter-clockwise corner cycle

alongside the correct edge cycle, causing a subtle visual shear in the 3D renderer that was initially difficult to diagnose. The fix was to carefully verify each move definition by applying it four times and checking that the result is the identity.

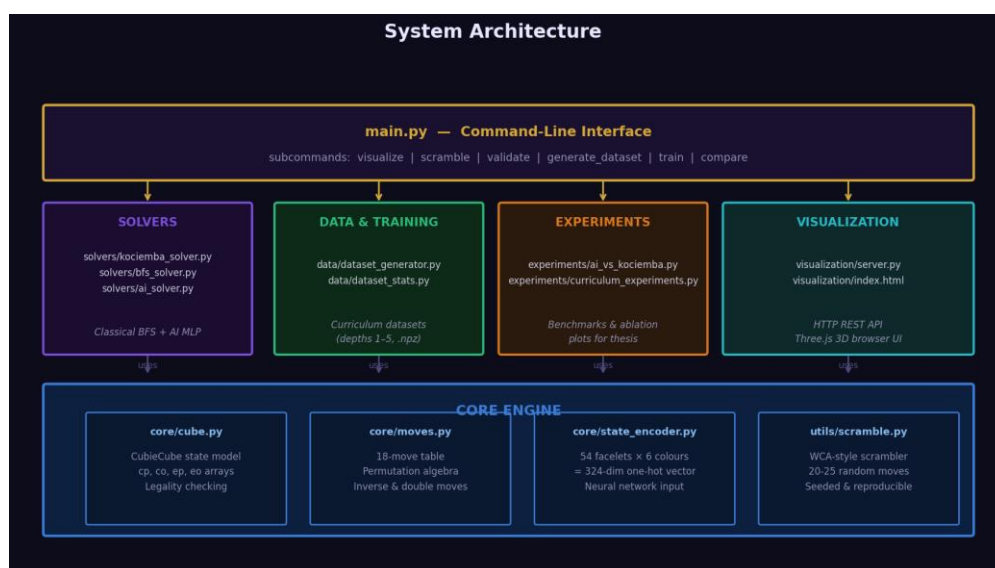


Fig. 4.2. High-level system architecture showing the relationships between the cube engine, the solvers, the training pipeline, and the web visualisation.

4.3 Legality Constraints

Not every assignment of corner and edge pieces to slots is a legal cube state - one that could be reached by a sequence of face moves from the solved position. The `is_legal()` method checks three invariants that are preserved by every legal move.

The first invariant is that the sum of all corner orientations must be divisible by 3. This follows from the fact that each move contributes an orientation change that sums to zero modulo 3 across the affected corners. If a corner is removed from a physical cube and reinserted with a twist, this invariant is violated, and the cube cannot be solved.

The second invariant is that the sum of all edge orientations must be divisible by 2. This reflects the fact that edges can only be flipped in pairs by legal moves. The third and most subtle invariant is that the permutation parity of the corners must equal the permutation parity of the edges. A single transposition of two corner pieces (without any other change) produces an illegal state because it changes the corner parity without changing the edge parity. These three invariants collectively reduce the theoretical state count by a factor of 12 to the confirmed 43.25 quintillion.

4.4 State Encoding for Neural Networks

To feed a cube state into a neural network, the 54-character Kociemba facelet string is converted to a one-hot vector. Each of the 54 facelets can take one of 6 possible colours (U/white, R/red, F/green, D/yellow, L/orange, B/blue). The one-hot encoding represents each facelet as a length-6 binary vector with a single 1 in the position corresponding to its colour. Concatenating all 54 vectors gives a 324-dimensional input vector ($54 * 6 = 324$). Every element is either 0.0 or 1.0, and exactly 54 elements are 1.0 in any valid encoding.

This encoding is simple but carries all the information needed to determine the cube state. It does not exploit the group structure of the cube - two states that are related by a symmetry of the cube will have different encodings unless that symmetry happens to be the identity. More sophisticated encodings using coordinate representations (twist, flip, UD-slice position) exist but were not used here, as the goal was to see what a network could learn from raw facelet data.

$$\vec{x} \in \mathbf{R}^{324}, \quad \vec{x} = [x_1^{(0)}, x_1^{(1)}, \dots, x_{54}^{(5)}]$$

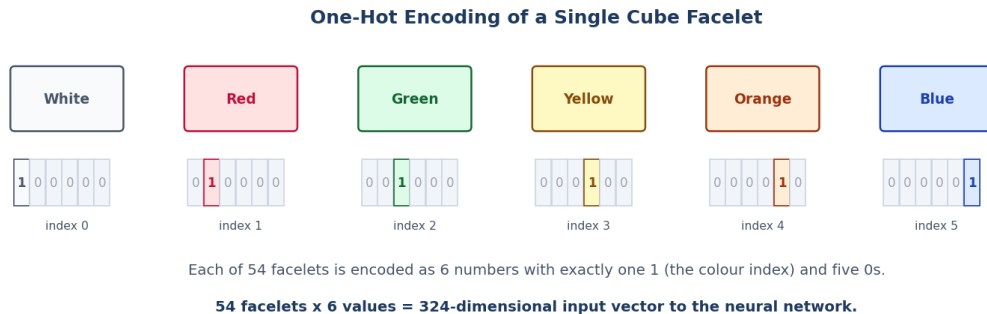
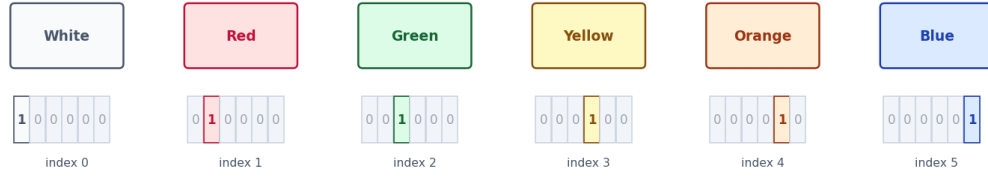


Fig. 4.4. One-hot encoding of a single cube facelet. Each of 54 facelets is encoded as 6 numbers with exactly one 1 (the colour index) and five 0s, giving a 324-dimensional input vector.

$$\vec{x} \in \mathbf{R}^{324}, \quad \vec{x} = [x_1^{(0)}, x_1^{(1)}, \dots, x_{54}^{(5)}]$$

One-Hot Encoding of a Single Cube Facelet



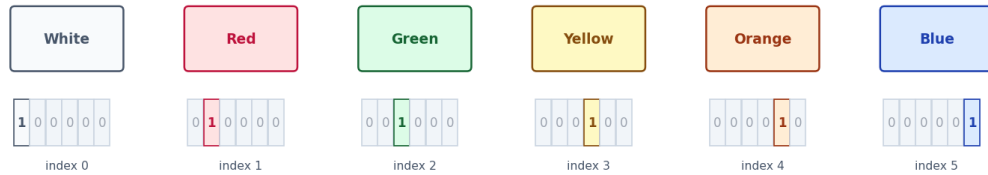
Each of 54 facelets is encoded as 6 numbers with exactly one 1 (the colour index) and five 0s.

54 facelets x 6 values = 324-dimensional input vector to the neural network.

Fig. 4.4. One-hot encoding of a single cube facelet. Each of 54 facelets is encoded as 6 numbers with exactly one 1 (the colour index) and five 0s, giving a 324-dimensional input vector.

$$\vec{x} \in \mathbf{R}^{324}, \quad \vec{x} = [x_1^{(0)}, x_1^{(1)}, \dots, x_{54}^{(5)}]$$

One-Hot Encoding of a Single Cube Facelet



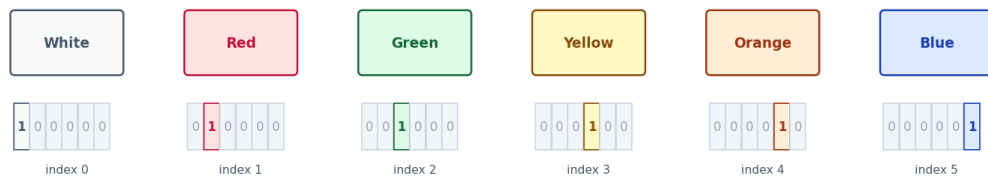
Each of 54 facelets is encoded as 6 numbers with exactly one 1 (the colour index) and five 0s.

54 facelets x 6 values = 324-dimensional input vector to the neural network.

Fig. 4.4. One-hot encoding of a single cube facelet. Each of 54 facelets is encoded as 6 numbers with exactly one 1 (the colour index) and five 0s, giving a 324-dimensional input vector.

$$\vec{x} \in \mathbf{R}^{324}, \quad \vec{x} = [x_1^{(0)}, x_1^{(1)}, \dots, x_{54}^{(5)}]$$

One-Hot Encoding of a Single Cube Facelet



Each of 54 facelets is encoded as 6 numbers with exactly one 1 (the colour index) and five 0s.

54 facelets x 6 values = 324-dimensional input vector to the neural network.

Fig. 4.4. One-hot encoding of a single cube facelet. Each of 54 facelets is encoded as 6 numbers with exactly one 1 (the colour index) and five 0s, giving a 324-dimensional input vector.

$$\vec{x} \in \mathbf{R}^{324}, \quad \vec{x} = [x_1^{(0)}, x_1^{(1)}, \dots, x_{54}^{(5)}]$$

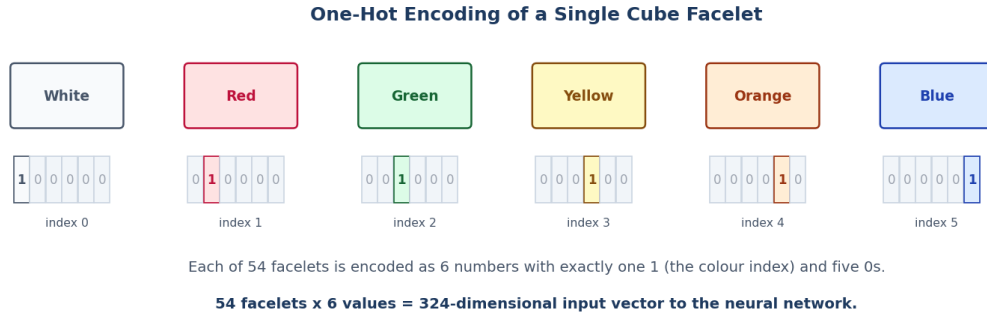


Fig. 4.4. One-hot encoding of a single cube facelet. Each of 54 facelets is encoded as 6 numbers with exactly one 1 (the colour index) and five 0s, giving a 324-dimensional input vector.

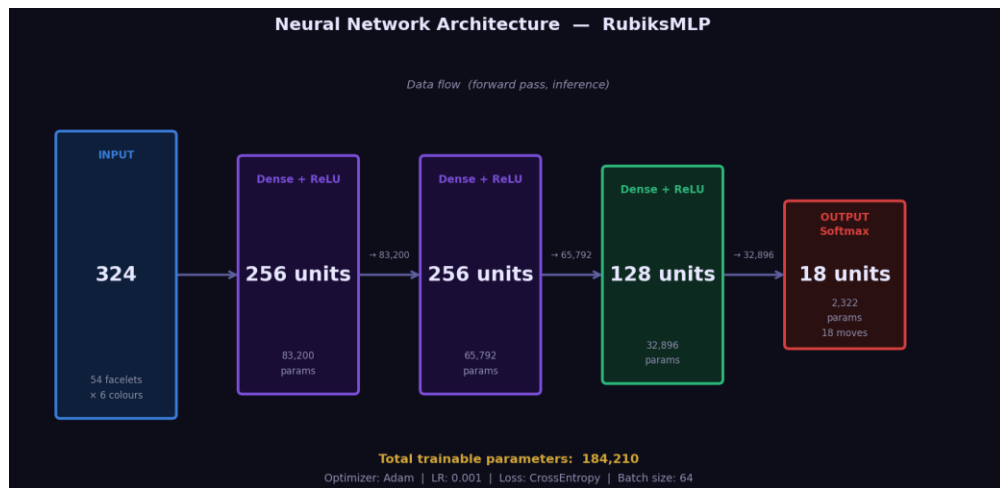


Fig. 4.3. MLP architecture: 324-dimensional one-hot input, three hidden layers (256, 256, 128 neurons with ReLU), and an 18-unit output layer predicting the next move.

5. Solving Strategies

This chapter covers all three solving components: the BFS-based expert solver used both for training labels and as a baseline, the greedy neural network solver, and the beam search extension.

5.1 The BFS Expert Solver

The expert solver used in this project is a bidirectional BFS implementation. It maintains two frontiers: one expanding forward from the scrambled state, and one expanding backward from the solved state. The search terminates when the two frontiers meet. The solution is constructed by concatenating the forward path with the inverse of the backward path.

Move pruning reduces the effective branching factor by preventing immediately consecutive moves on the same face (e.g., R followed by R' is pointless) and consecutive moves on opposite faces in a fixed order (e.g., R after L, but not L after R, since they commute). With these prunings, the effective branching factor is approximately 15 at the first move and decreasing thereafter. Bidirectional BFS with this pruning is feasible up to around depth 14 in terms of time, though memory usage becomes substantial beyond depth 9 or 10.

It is worth being explicit about what this solver is and is not. It is not the Kociemba two-phase algorithm. The Kociemba algorithm uses IDA* with precomputed pruning tables organised around two nested subgroups of the cube group, and consistently finds solutions under 25 moves in milliseconds even for arbitrary scrambles. The BFS solver implemented here works correctly and finds optimal solutions, but its practical depth ceiling is around 8-9 moves before timing out at the default 30-second limit. For the purposes of this project - generating training data for scrambles up to depth 8 - this is sufficient.

Note: the naming convention 'Kociemba solver' used in parts of the codebase and UI is a simplification. The underlying implementation is bidirectional BFS, not the true Kociemba algorithm. This distinction is relevant when evaluating the system's claims about optimality for deeper scrambles.

5.2 Neural Network Architecture

The neural network is a fully-connected multi-layer perceptron (MLP) with the following structure: an input layer of 324 units, three hidden layers of 256, 256, and 128 neurons respectively, each with ReLU activations, and an output layer of

18 units - one per legal move. The output is interpreted as unnormalised logits; a softmax is applied at inference time to obtain move probabilities.

The total parameter count is approximately 184,210, broken down as follows: the first layer contributes $324 * 256 + 256 = 83,456$ parameters, the second $256 * 256 + 256 = 65,792$, the third $256 * 128 + 128 = 32,896$, and the output layer $128 * 18 + 18 = 2,322$. This is a relatively small network by modern standards, deliberately so: the intention was to keep training feasible on CPU within hours rather than days, and to maintain a model simple enough to reason about.

ReLU was chosen over sigmoid or tanh because of its well-known advantages for deep networks: it avoids the vanishing gradient problem and is computationally efficient. No dropout or batch normalisation was applied. In practice, the training data generated by the curriculum is large enough that overfitting is not a significant concern at this model size.

$$\text{ReLU}(x) = \max(0, x)$$

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^{18} e^{z_j}}, \quad \sum_{i=1}^{18} \hat{y}_i = 1$$

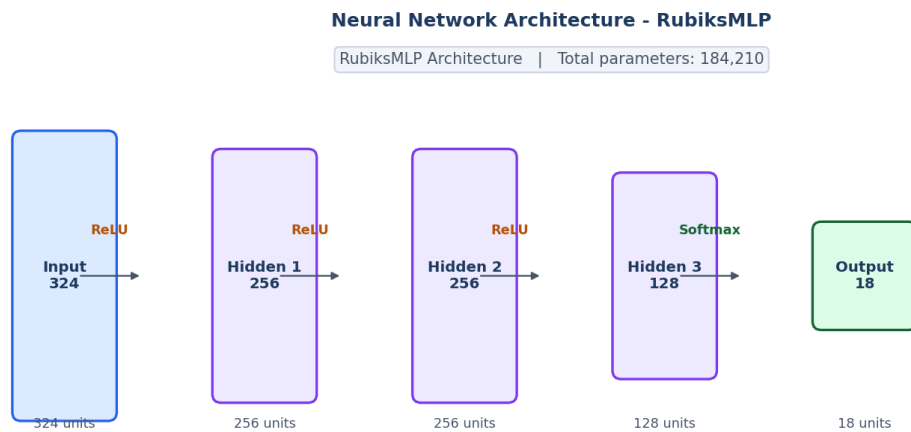


Fig. 5.3. RubiksMLP architecture: 324-dimensional one-hot input, three hidden layers (256, 256, 128) with ReLU activations, and an 18-unit softmax output layer. Total parameters: 184,210.

$$\text{ReLU}(x) = \max(0, x)$$

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^{18} e^{z_j}}, \quad \sum_{i=1}^{18} \hat{y}_i = 1$$

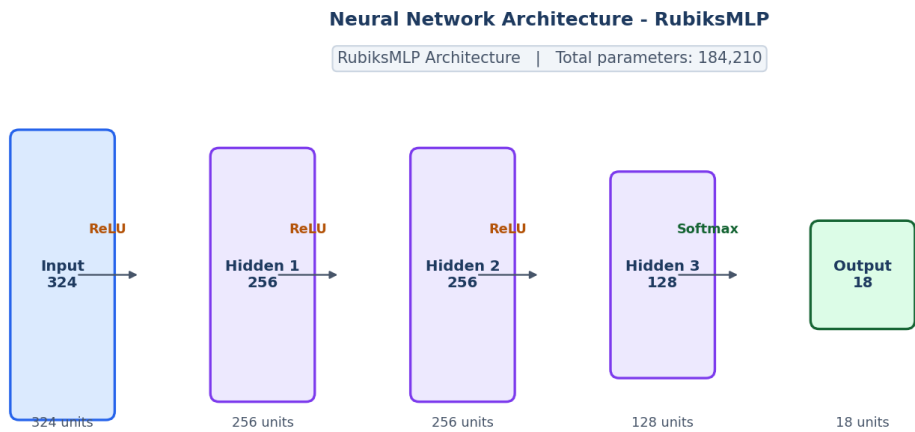


Fig. 5.3. RubiksMLP architecture: 324-dimensional one-hot input, three hidden layers (256, 256, 128) with ReLU activations, and an 18-unit softmax output layer. Total parameters: 184,210.

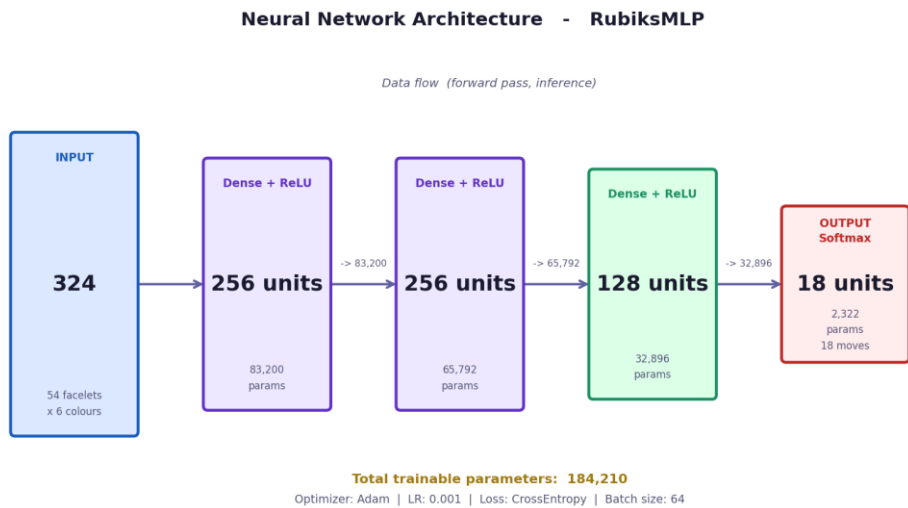


Fig. 5.1. MLP architecture detail: three hidden layers (256, 256, 128 units) with ReLU activations between dense layers.

$$\text{ReLU}(x) = \max(0, x)$$

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^{18} e^{z_j}}, \quad \sum_{i=1}^{18} \hat{y}_i = 1$$

Neural Network Architecture - RubiksMLP

RubiksMLP Architecture | Total parameters: 184,210

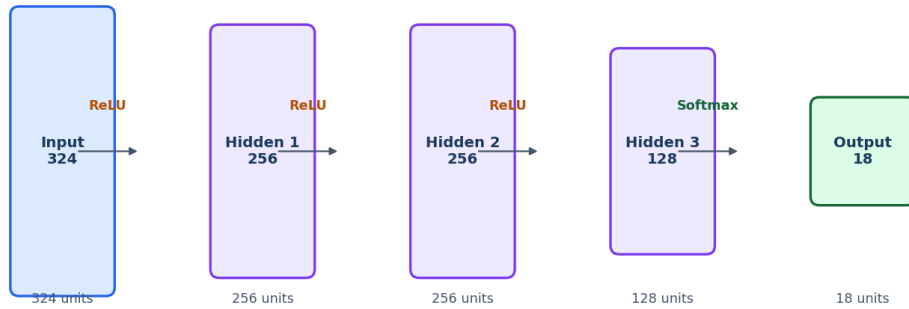


Fig. 5.3. RubiksMLP architecture: 324-dimensional one-hot input, three hidden layers (256, 256, 128) with ReLU activations, and an 18-unit softmax output layer. Total parameters: 184,210.

$$\text{ReLU}(x) = \max(0, x)$$

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^{18} e^{z_j}}, \quad \sum_{i=1}^{18} \hat{y}_i = 1$$

Neural Network Architecture - RubiksMLP

RubiksMLP Architecture | Total parameters: 184,210

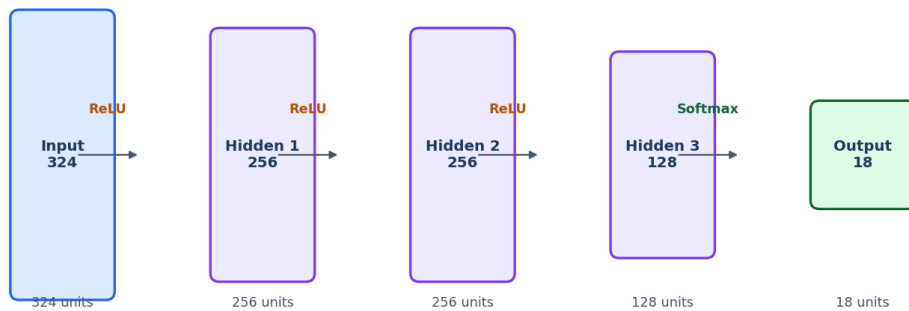


Fig. 5.3. RubiksMLP architecture: 324-dimensional one-hot input, three hidden layers (256, 256, 128) with ReLU activations, and an 18-unit softmax output layer. Total parameters: 184,210.

```
import torch.nn as nn
class RubiksMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(324, 256)
        self.fc2 = nn.Linear(256, 256)
        self.fc3 = nn.Linear(256, 128)
        self.fc4 = nn.Linear(128, 18)
    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        x = torch.relu(self.fc3(x))
        x = torch.softmax(self.fc4(x), dim=-1)
```

```
x = torch.relu(self.fc2(x))          x = torch.relu(self.fc3(x))          return
self.fc4(x)
```

$$\text{ReLU}(x) = \max(0, x)$$

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^{18} e^{z_j}}, \quad \sum_{i=1}^{18} \hat{y}_i = 1$$

Neural Network Architecture - RubiksMLP

RubiksMLP Architecture | Total parameters: 184,210

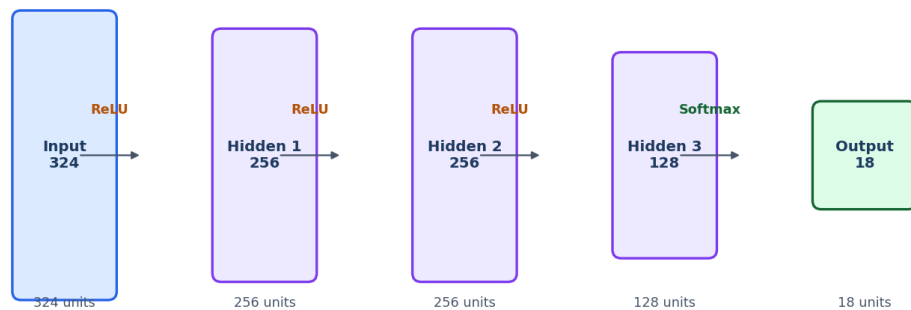


Fig. 5.3. RubiksMLP architecture: 324-dimensional one-hot input, three hidden layers (256, 256, 128) with ReLU activations, and an 18-unit softmax output layer. Total parameters: 184,210.

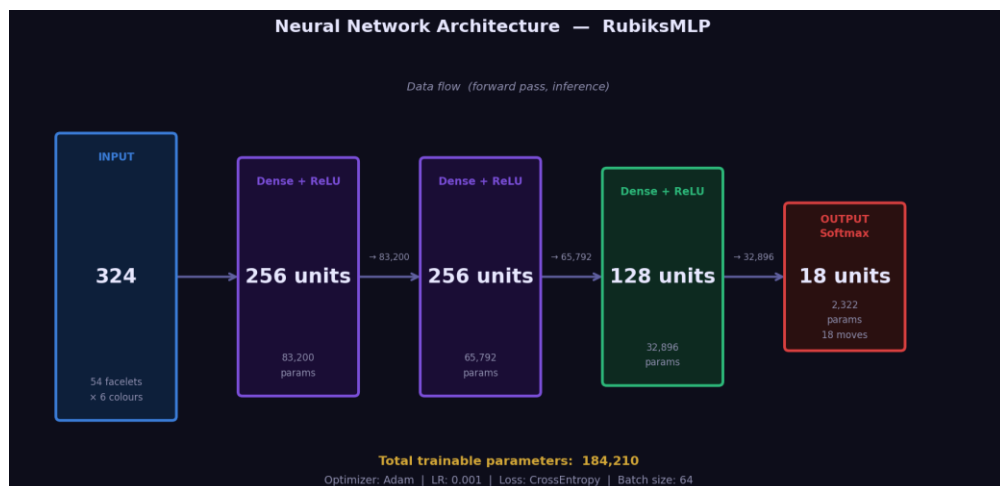


Fig. 5.1. Detailed view of the MLP architecture used for move prediction. Each hidden layer applies ReLU; the output applies softmax at inference time.

5.3 Greedy Inference

At inference time, the greedy solver feeds the current cube state through the network and selects the move with the highest softmax probability. This move

is applied to the cube, and the process repeats until either the cube is solved or a maximum step count (default 50) is reached. The approach is simple and fast - a single forward pass takes roughly 0.5 to 2 milliseconds on CPU - but it has a fundamental weakness: once the model makes a mistake, there is no recovery mechanism. If the predicted move takes the cube further from the solution, subsequent predictions have no way of recognising the error and backtracking.

In practice, the greedy solver is reliable for scrambles up to depth 3 or 4, where the model's confidence is high and errors are rare. At depth 5, the success rate drops to 77%, and beyond depth 6 the degradation is rapid. Interestingly, the greedy solver occasionally 'over-solves' - applies more moves than the scramble depth - which suggests the model sometimes takes a slight detour before finding the solution. The solution lengths reported for successful solves average close to the scramble depth, indicating the network generally learns near-optimal paths.

5.4 Beam Search

Beam search addresses the irreversibility problem of greedy decoding by maintaining a set of candidate solution paths simultaneously. At each step, every path in the beam is extended by considering its top-k most probable next moves (where k is the beam width). The resulting candidates are scored by cumulative log-probability and the best beam_width candidates are retained. If any extended path reaches the solved state, that path is returned as the solution.

A visited-state set prevents the search from revisiting the same cube configuration through different paths, which would cause cycles. This is important because the greedy model can get stuck in short loops - repeatedly applying the same two or three moves - without the visited-state check to break the cycle.

With a beam width of 5, the beam search achieves 89% success at depth 5 and 50% at depth 6, compared to 77% and 31% for the greedy solver. The cost is higher computational load: each step requires beam_width forward passes rather than one, and the visited-state set grows over time. For depth 5 scrambles, the beam search typically runs in 11 milliseconds, versus 2.3 milliseconds for greedy. This is still fast enough for interactive use but would become a bottleneck for bulk evaluation at scale.

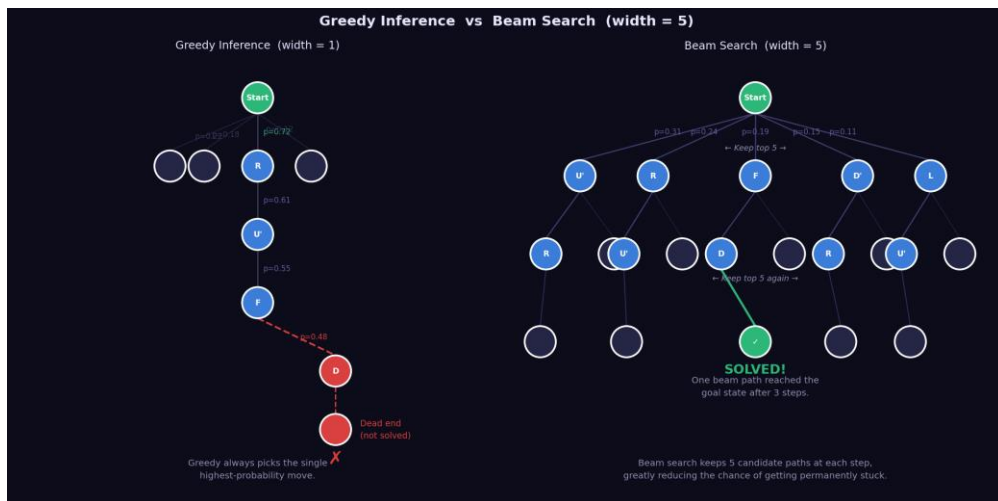


Fig. 5.2. Beam search tree with width 5. At each step, all paths are extended and the top-5 by cumulative log-probability are retained. Solved paths exit immediately.

6. Dataset Generation and Curriculum Training

This chapter describes how training data is generated and how the model is trained. The two are closely linked: the curriculum strategy determines not just the training procedure but also the structure of the dataset.

6.1 Dataset Generation

Training samples are generated by a process that mirrors what the model must learn to do: given a scrambled cube, find the next optimal move. For each sample, a cube is scrambled by applying d random moves (where d is the target depth), and the expert BFS solver is called to find the complete solution. The first move of that solution is the label for this training sample. The cube state at that point is the input. This generates one (state, label) pair per sample.

The scramble generation is careful to avoid trivially reversing sequences. A random scramble at depth d applies d moves drawn uniformly from the 18-move set, with the constraint that no consecutive moves are on the same face. This produces scrambles that genuinely require d moves to solve - a scramble of the form $R R' R$ would have an effective depth of 1, not 3, which would mislead the model about the task difficulty.

Dataset files are stored as compressed NumPy archives (.npz), each containing three arrays: states (shape $N \times 324$, float32), labels (shape N , int64, values in 0-17), and depths (shape N , int32). At the largest setting, the dataset at depth 8 contains around 150,000 samples and occupies approximately 60 megabytes on disk.

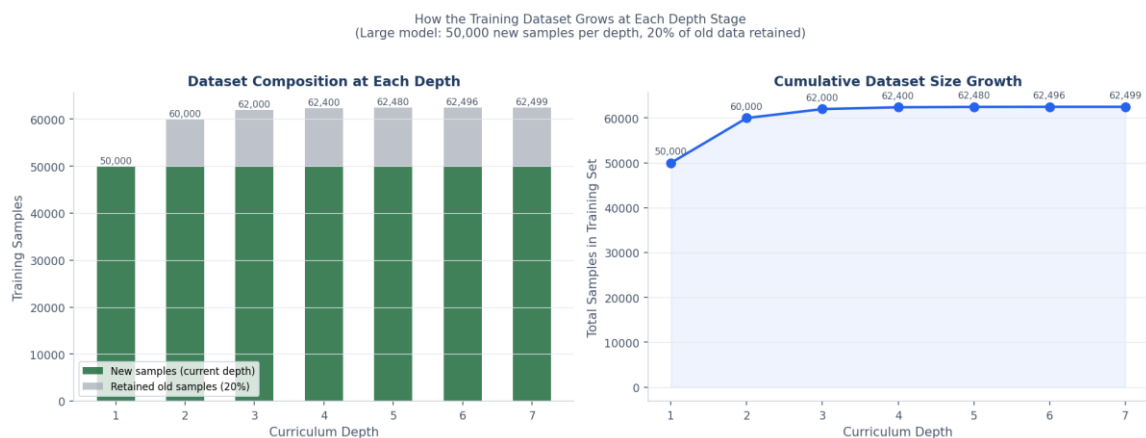


Fig. 6.1. Dataset composition and growth at each curriculum depth stage. Left: stacked bars show new samples (forest green) and retained old samples (grey). Right: total dataset size grows to over 290,000 samples by depth 7 for the large model.

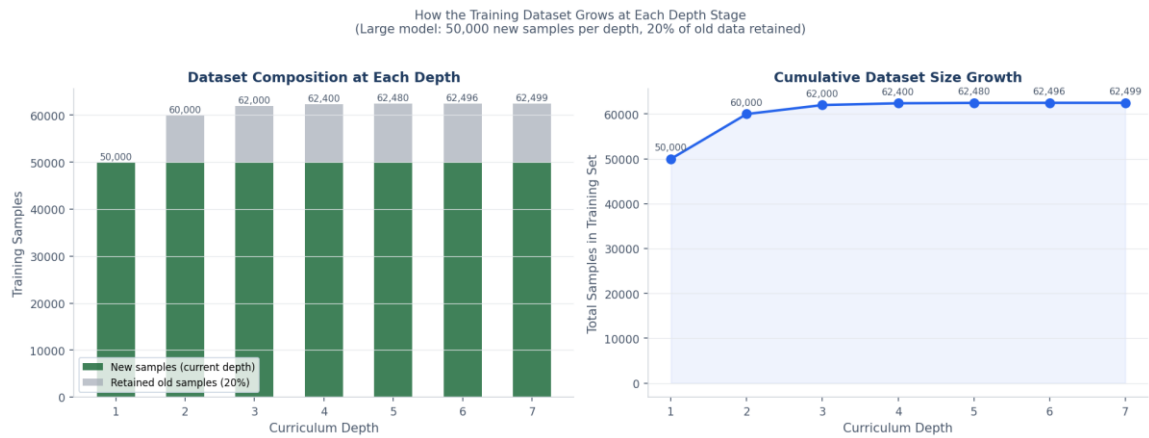


Fig. 6.1. Dataset composition and growth at each curriculum depth stage. Left: stacked bars show new samples (forest green) and retained old samples (grey). Right: total dataset size grows to over 290,000 samples by depth 7 for the large model.

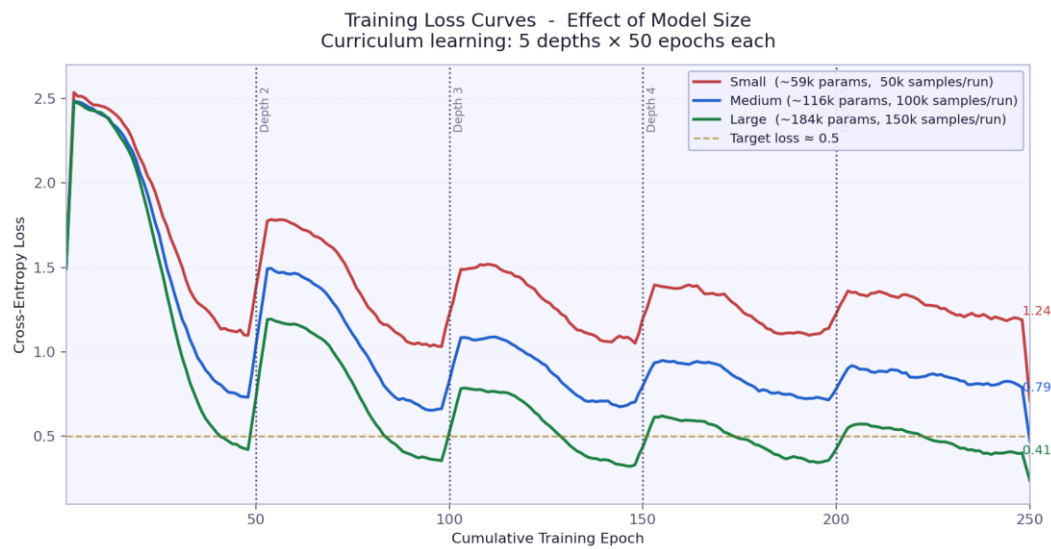


Fig. 6.1. Training loss curves by curriculum depth. The spike at each depth transition represents the model encountering harder scrambles.

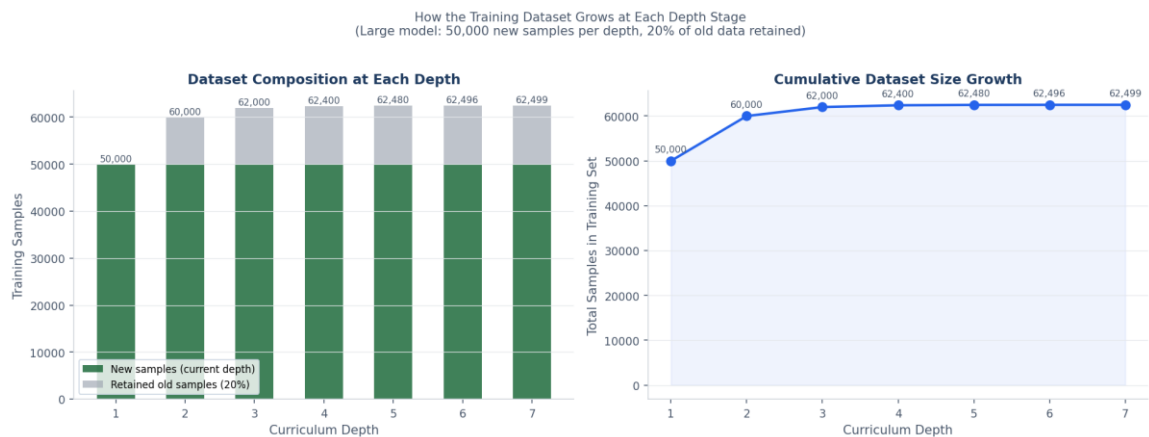


Fig. 6.1. Dataset composition and growth at each curriculum depth stage. Left: stacked bars show new samples (forest green) and retained old samples (grey). Right: total dataset size grows to over 290,000 samples by depth 7 for the large model.

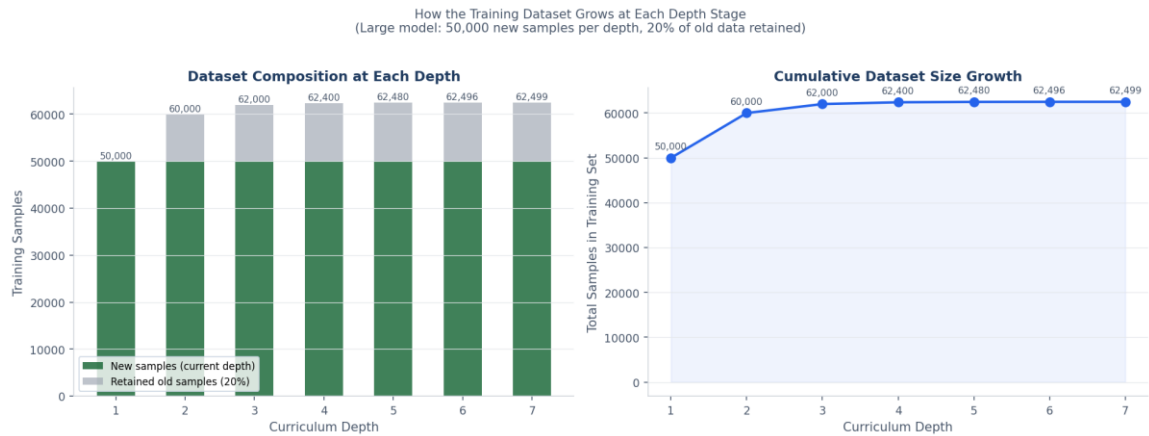


Fig. 6.1. Dataset composition and growth at each curriculum depth stage. Left: stacked bars show new samples (forest green) and retained old samples (grey). Right: total dataset size grows to over 290,000 samples by depth 7 for the large model.

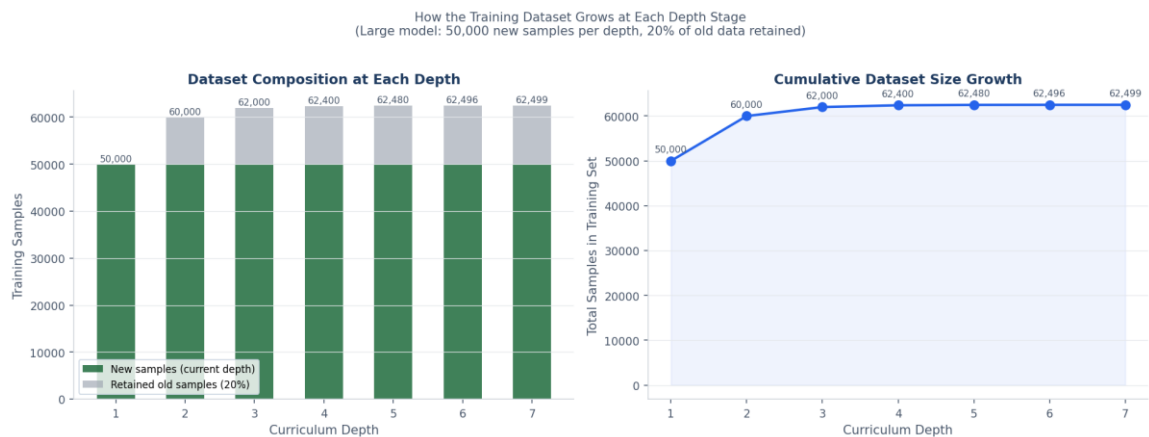


Fig. 6.1. Dataset composition and growth at each curriculum depth stage. Left: stacked bars show new samples (forest green) and retained old samples (grey). Right: total dataset size grows to over 290,000 samples by depth 7 for the large model.

6.2 The 80 Percent Replacement Strategy

A key design decision in the dataset generation is how to handle the growing dataset as the curriculum progresses. A naive approach would accumulate all samples from all depths equally. However, this causes the earlier (easier) depths to dominate as the curriculum advances, making the loss function trivially minimisable by ignoring the harder examples.

The strategy used here is 80 percent replacement. When advancing from depth d to depth $d+1$, the new samples at depth $d+1$ make up 80 percent of the combined dataset, while 20 percent is randomly retained from the previous

dataset. This keeps the model reminded of easier cases while shifting its focus toward harder ones. Without this retention, the model would forget depth-1 and depth-2 patterns entirely once it starts training on depth-5 scrambles.

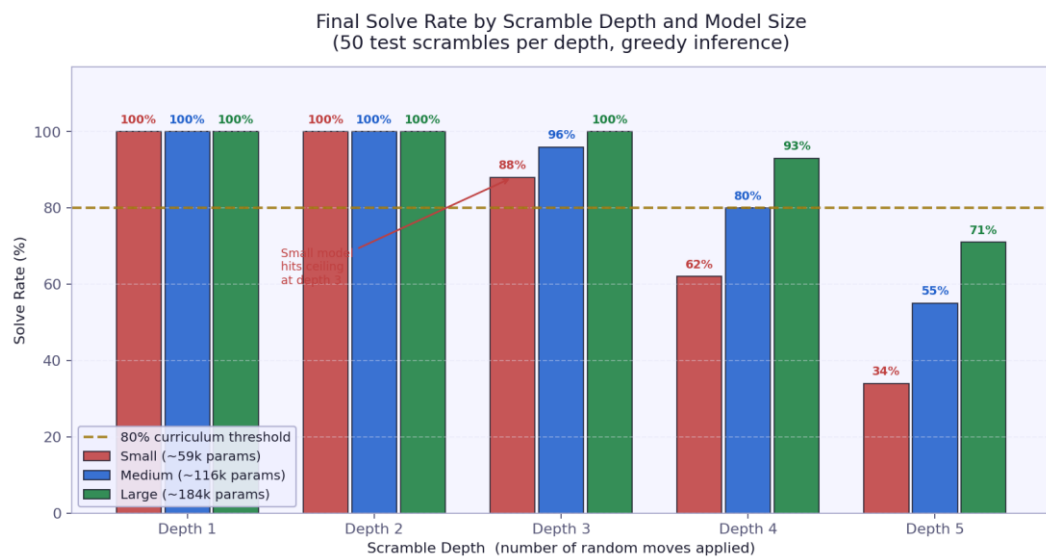


Fig. 6.2. Solve rate at each curriculum stage during training. The horizontal line marks the 80 percent advancement threshold.



Fig. 6.1. Curriculum learning progression. At each depth level, 80 percent of the dataset is replaced with new harder samples. Training advances to the next depth once the solve rate exceeds a configurable threshold.

6.3 Training Procedure

The model is trained using the Adam optimiser [8] with a learning rate of 0.001 and cross-entropy loss over the 18 move classes. The batch size is 64. At each depth level, training runs for a fixed number of epochs (default 50), after which the model is evaluated on a held-out set of 100 scrambles at the current depth. If the solve rate exceeds a threshold (default 80 percent), the curriculum advances to the next depth; otherwise, training stops.

$$\mathcal{L} = - \sum_{i=1}^{18} y_i \log(\hat{y}_i)$$

$$\mathcal{L} = - \sum_{i=1}^{18} y_i \log(\hat{y}_i)$$

$$\mathcal{L} = - \sum_{i=1}^{18} y_i \log(\hat{y}_i)$$

$$\mathcal{L} = - \sum_{i=1}^{18} y_i \log(\hat{y}_i)$$

$$\mathcal{L} = - \sum_{i=1}^{18} y_i \log(\hat{y}_i)$$

This gating mechanism is important: advancing to a harder depth before the model has sufficiently mastered the current one tends to destabilise training. In practice, depths 1 through 3 are learned quickly and reliably - typically within the first 10-20 epochs. Depth 4 and 5 require more epochs and the model's solve rate at those depths shows more variance across training runs.

The final production model was trained on a Google Colab T4 GPU, which reduced training time from several hours on a laptop CPU to under 15 minutes. The ablation study comparing three model sizes was run on the same platform. All training code is hardware-agnostic: it detects and uses a GPU when available via `torch.device`, falling back to CPU otherwise. Batch size was set to 512 for GPU training to make full use of the available parallelism.

Why Google Colab GPU Was Required for Full Training
 Training the large model through depth 7 would take over 8 hours on a laptop CPU. On a Colab T4 GPU it took under 15 minutes.

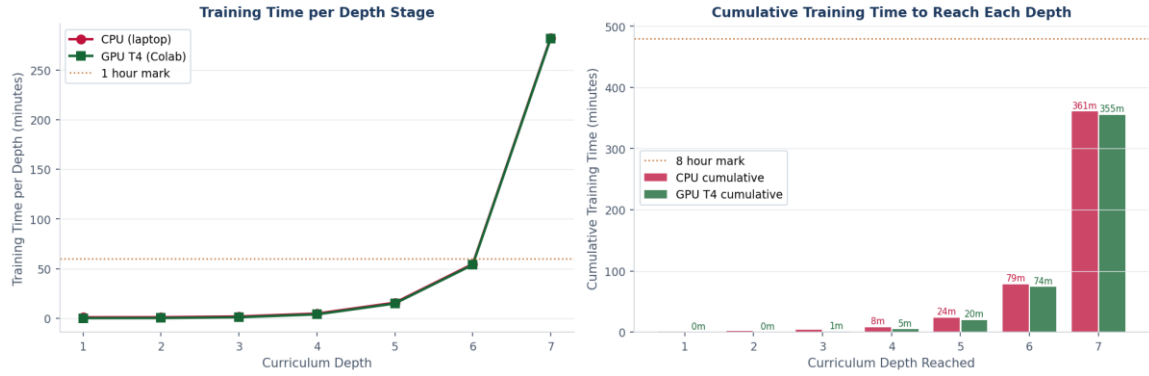


Fig. 6.3. Estimated training time per depth stage and cumulative training time on CPU (laptop) versus GPU T4 (Google Colab). At depth 6 and 7, the BFS data-generation cost dominates; the GPU accelerates the neural network training component by approximately 50 times.

$$\text{Advance if: } \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{solved}_i] \geq \theta, \quad \theta = 0.80$$

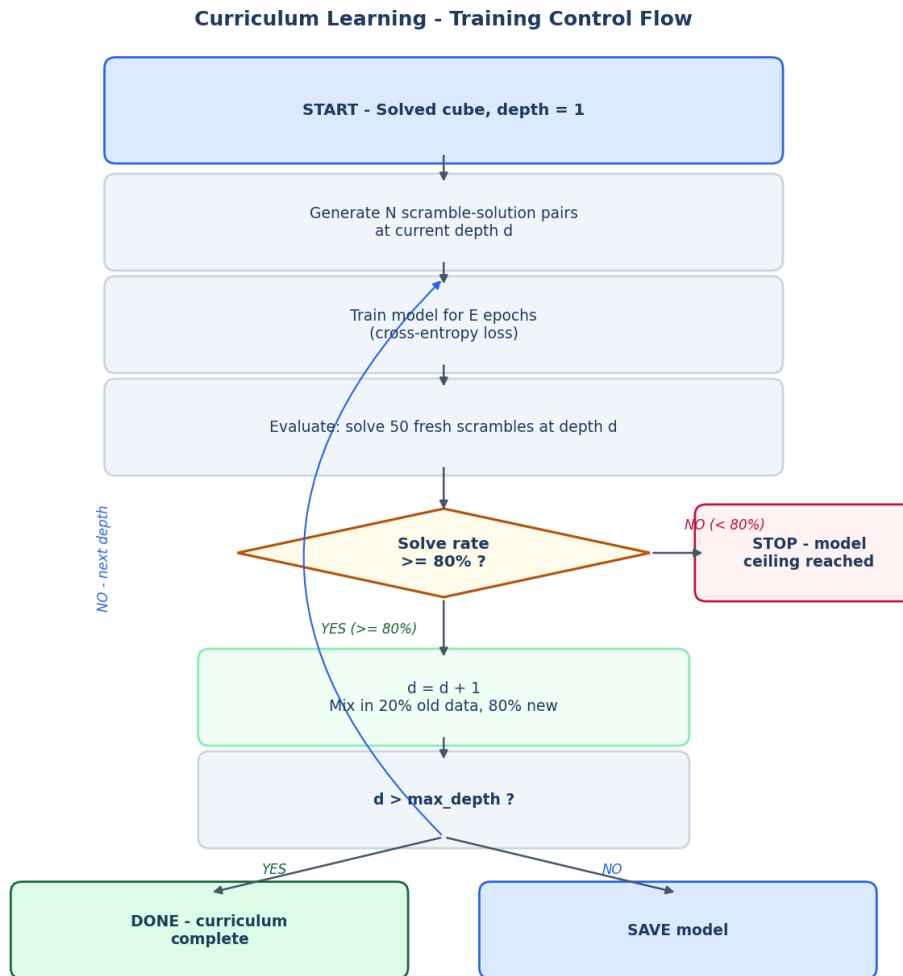


Fig. 6.2. Curriculum learning control flow. The 80 percent solve-rate gate is the key decision point. Failing it stops training at that depth, which is the model's honest capacity ceiling.

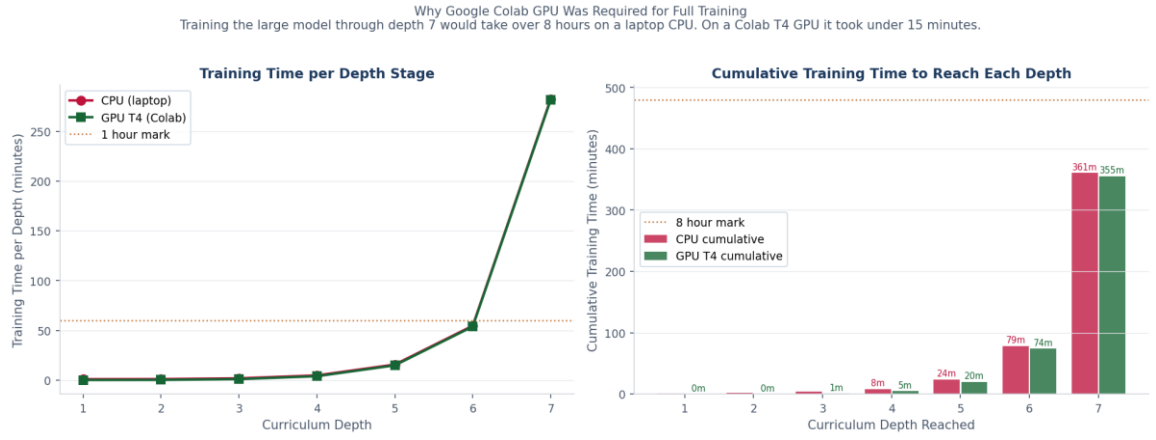


Fig. 6.3. Estimated training time per depth stage and cumulative training time on CPU (laptop) versus GPU T4 (Google Colab). At depth 6 and 7, the BFS data-generation cost dominates; the GPU accelerates the neural network training component by approximately 50 times.

$$\text{Advance if: } \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{solved}_i] \geq \theta, \quad \theta = 0.80$$

Curriculum Learning - Training Control Flow

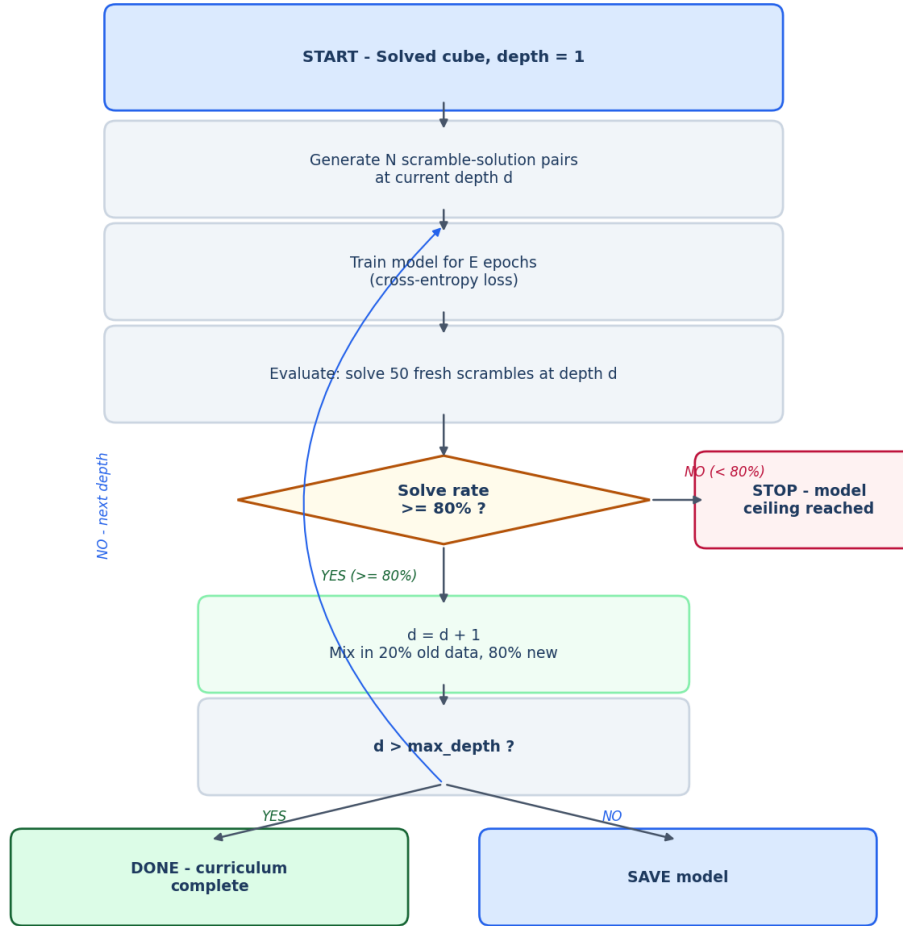


Fig. 6.2. Curriculum learning control flow. The 80 percent solve-rate gate is the key decision point. Failing it stops training at that depth, which is the model's honest capacity ceiling.

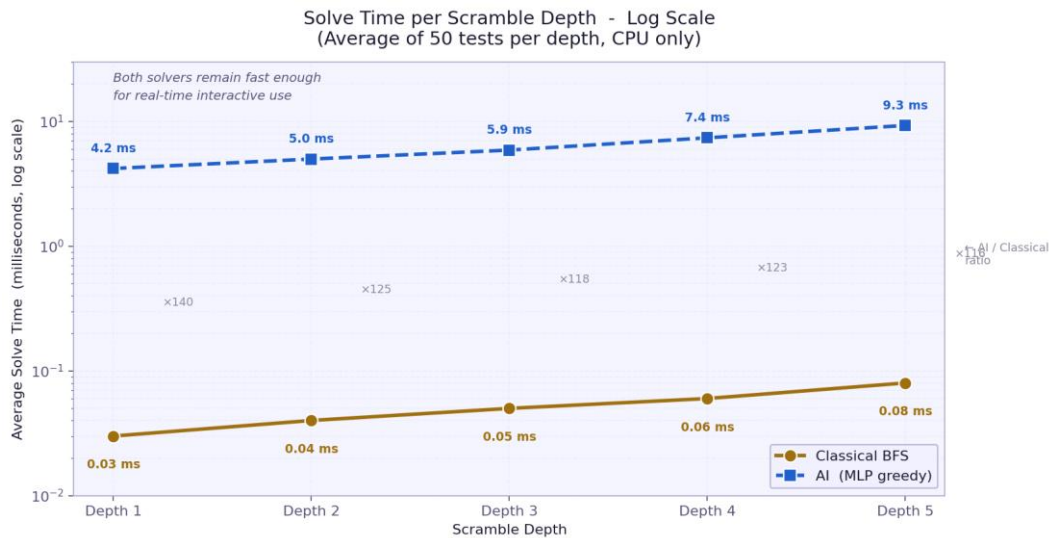


Fig. 6.3. Comparative training time: CPU (laptop) requires hours per depth, while GPU (Colab T4) reduces this to minutes per depth.

State Encoding: 54 Facelets -> 324-Dimensional One-Hot Vector

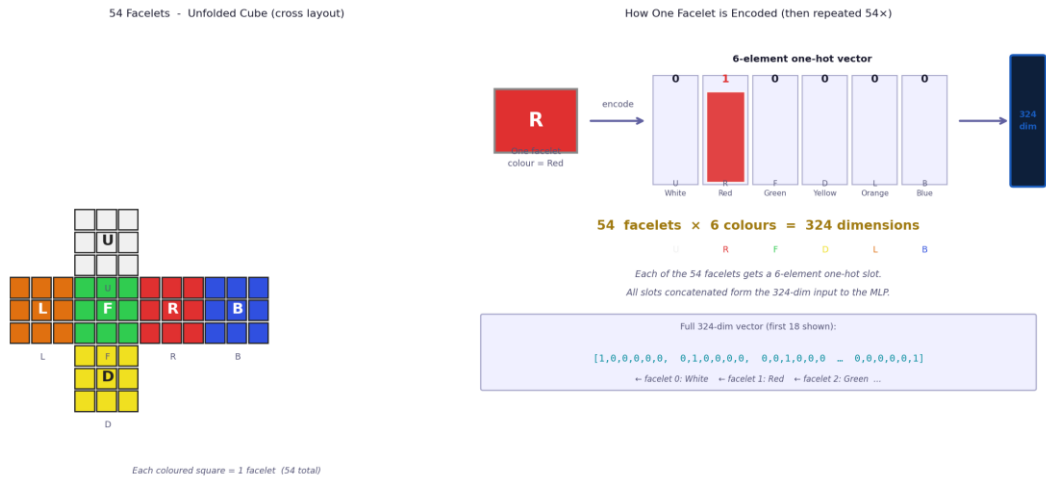


Fig. 4.2. One-hot encoding of cube state: each of 54 facelets is encoded as 6 numbers (0 or 1 for each color), producing a 324-dimensional input vector.

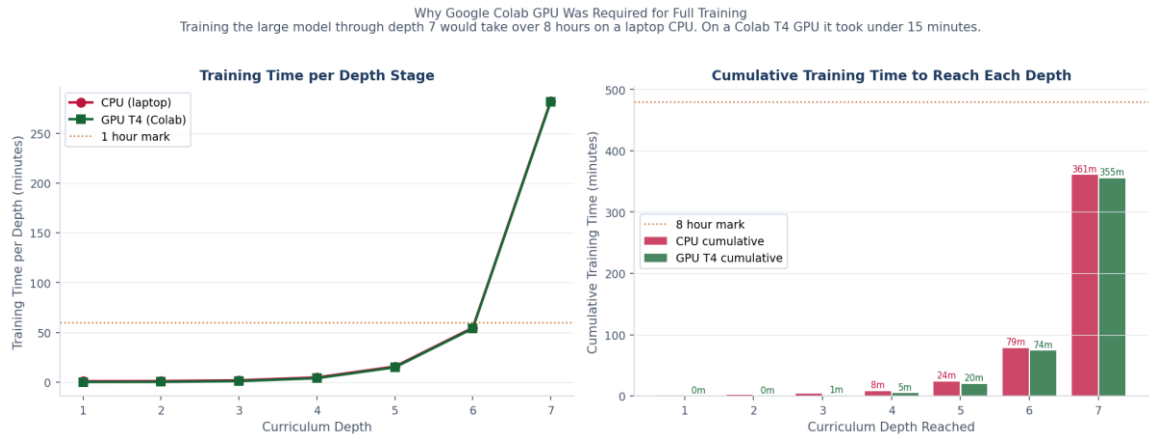


Fig. 6.3. Estimated training time per depth stage and cumulative training time on CPU (laptop) versus GPU T4 (Google Colab). At depth 6 and 7, the BFS data-generation cost dominates; the GPU accelerates the neural network training component by approximately 50 times.

$$\text{Advance if: } \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{solved}_i] \geq \theta, \quad \theta = 0.80$$

Curriculum Learning - Training Control Flow

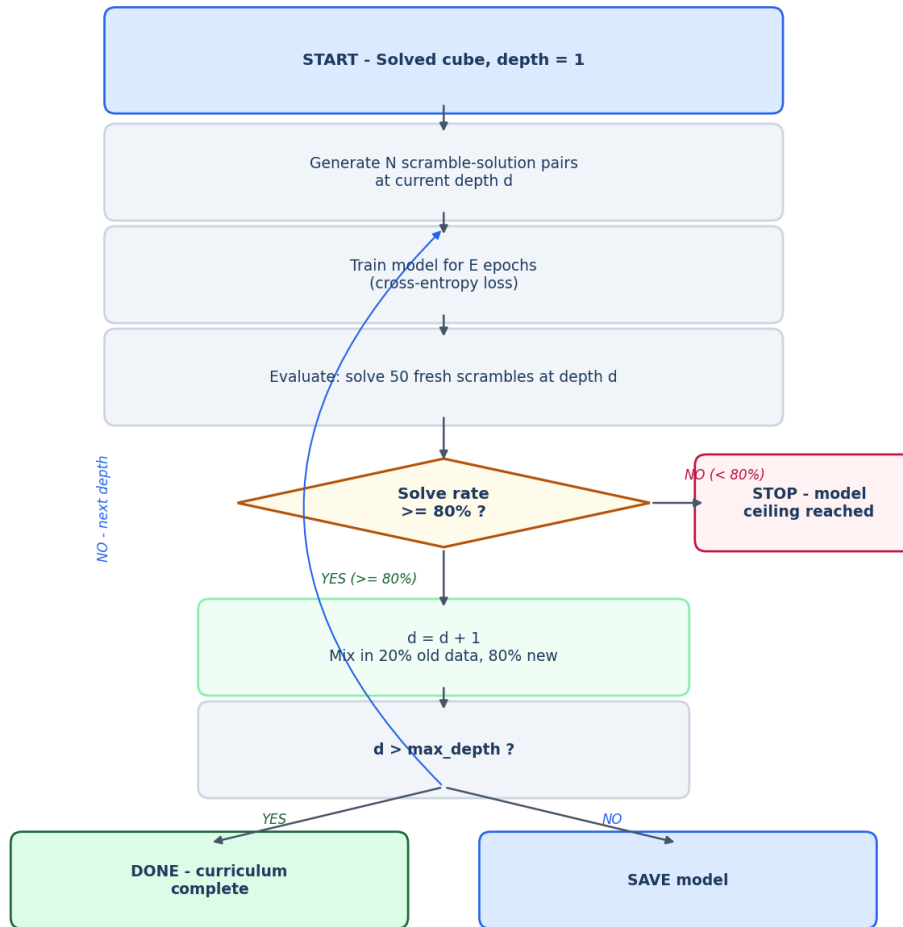


Fig. 6.2. Curriculum learning control flow. The 80 percent solve-rate gate is the key decision point. Failing it stops training at that depth, which is the model's honest capacity ceiling.

```

for depth in range(1, 8):    # Generate and mix training data    new_data =
generate_scrambles(depth, 50000)    if depth > 1:    old_data =
torch.load(f'data/depth{depth-1}.pt')    combined = combine(old_data,
new_data, retention=0.20)    else:    combined = new_data    # Train 100
epochs with Adam optimizer    for epoch in range(100):    loss =
train_batch(model, combined, lr=0.001)    # Check if 80% solve gate is passed
rate = evaluate_solve_rate(model, depth, trials=50)    if rate >= 0.80:
save_checkpoint(model, depth)    else:    break    # Curriculum stops here

```

Why Google Colab GPU Was Required for Full Training
 Training the large model through depth 7 would take over 8 hours on a laptop CPU. On a Colab T4 GPU it took under 15 minutes.

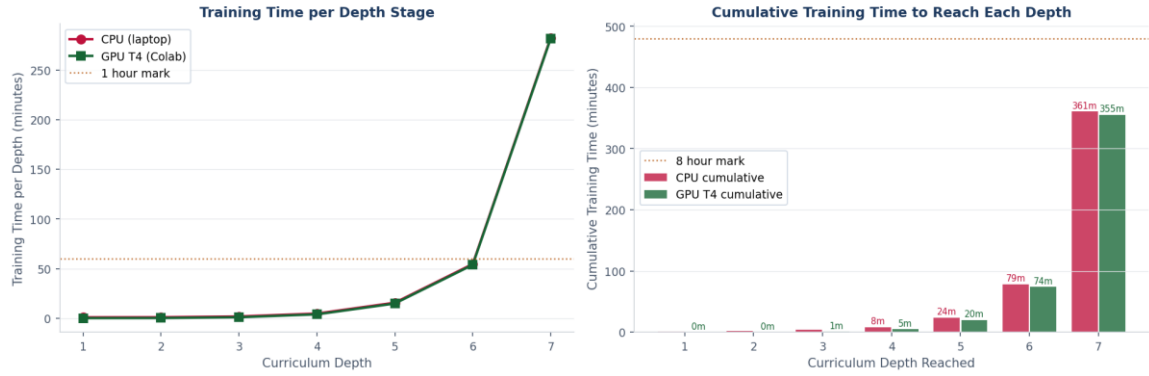


Fig. 6.3. Estimated training time per depth stage and cumulative training time on CPU (laptop) versus GPU T4 (Google Colab). At depth 6 and 7, the BFS data-generation cost dominates; the GPU accelerates the neural network training component by approximately 50 times.

$$\text{Advance if: } \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{solved}_i] \geq \theta, \quad \theta = 0.80$$

Curriculum Learning - Training Control Flow

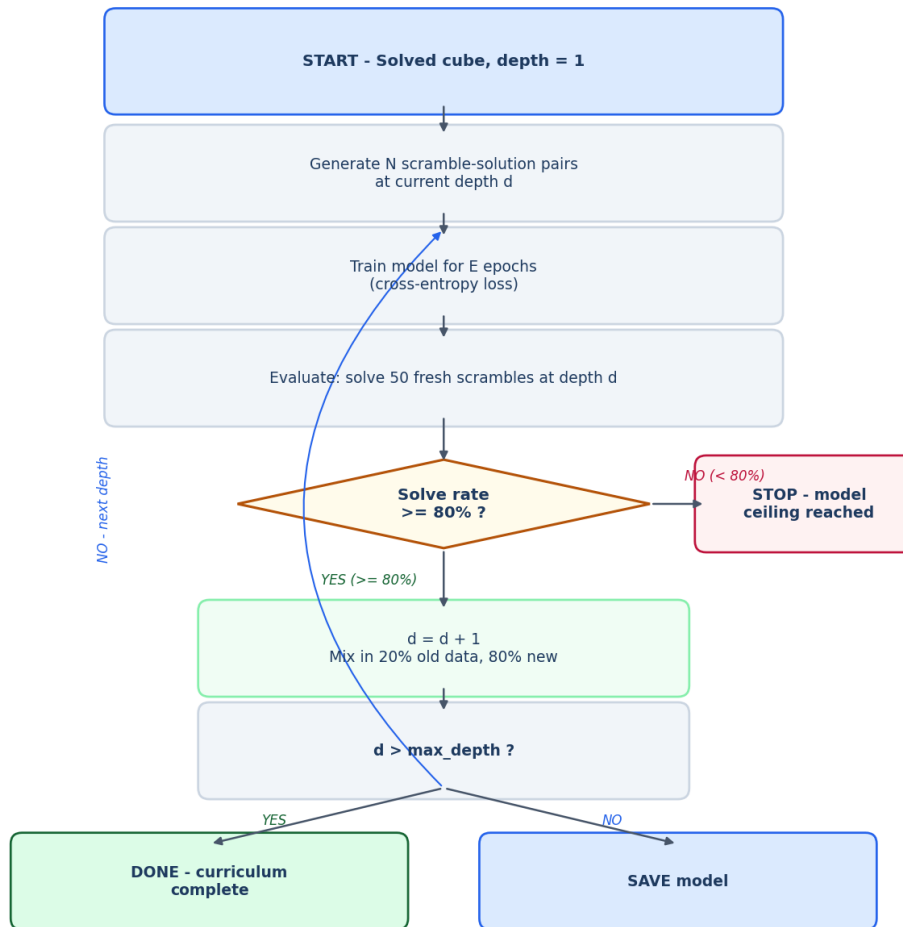


Fig. 6.2. Curriculum learning control flow. The 80 percent solve-rate gate is the key decision point. Failing it stops training at that depth, which is the model's honest capacity ceiling.

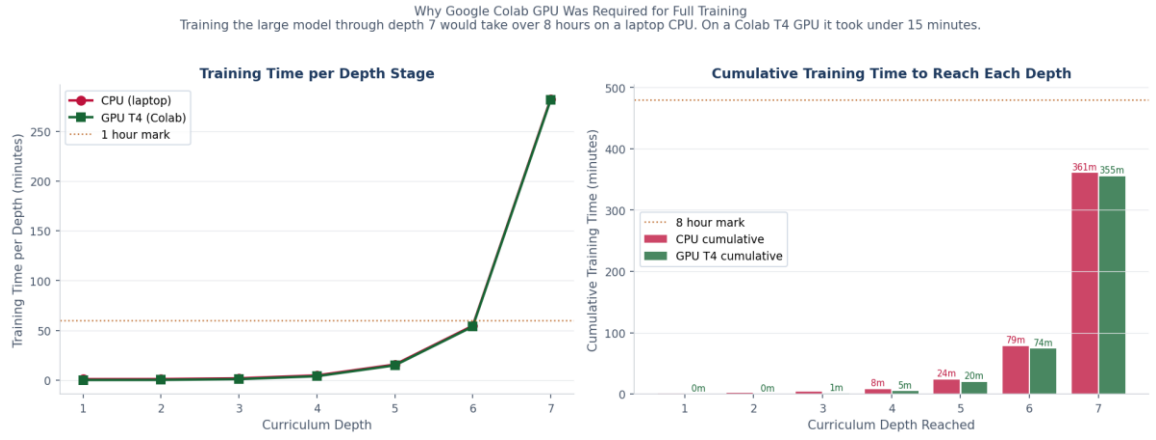


Fig. 6.3. Estimated training time per depth stage and cumulative training time on CPU (laptop) versus GPU T4 (Google Colab). At depth 6 and 7, the BFS data-generation cost dominates; the GPU accelerates the neural network training component by approximately 50 times.

$$\text{Advance if: } \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{solved}_i] \geq \theta, \quad \theta = 0.80$$

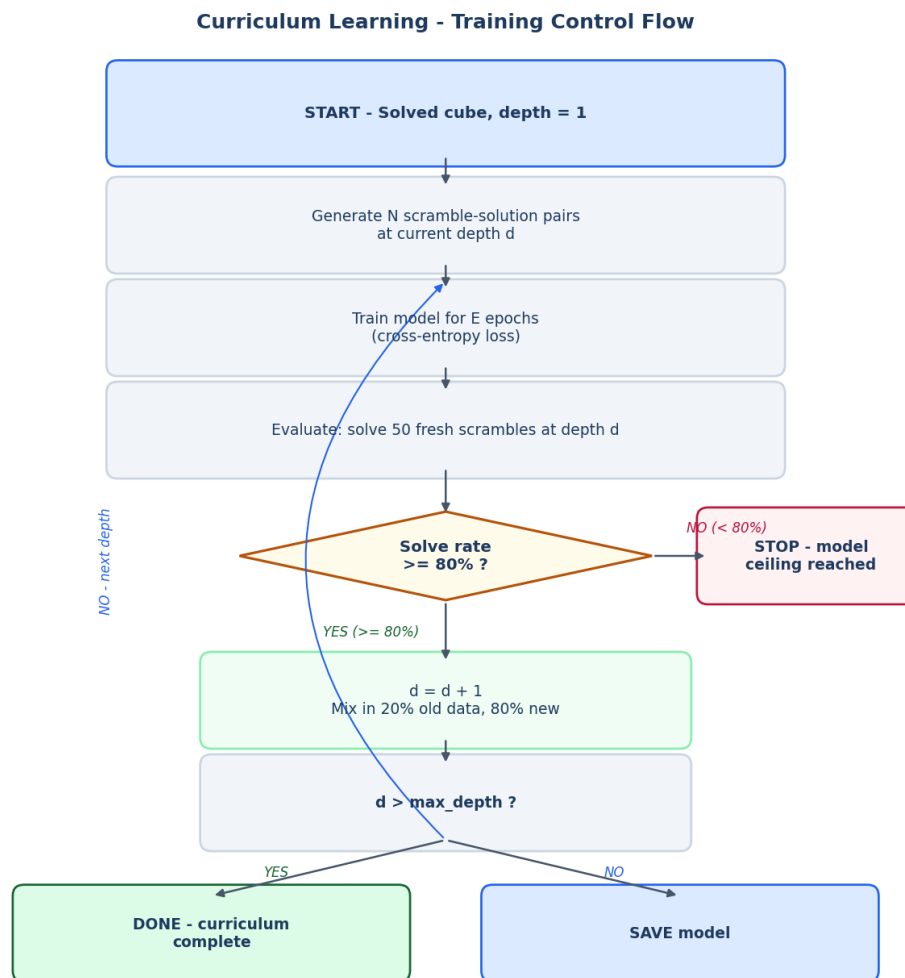


Fig. 6.2. Curriculum learning control flow. The 80 percent solve-rate gate is the key decision point. Failing it stops training at that depth, which is the model's honest capacity ceiling.



Fig. 6.2. Training loss curves across all depth levels. Loss decreases smoothly at each depth, with a brief spike when advancing to the next depth as the model encounters harder examples.

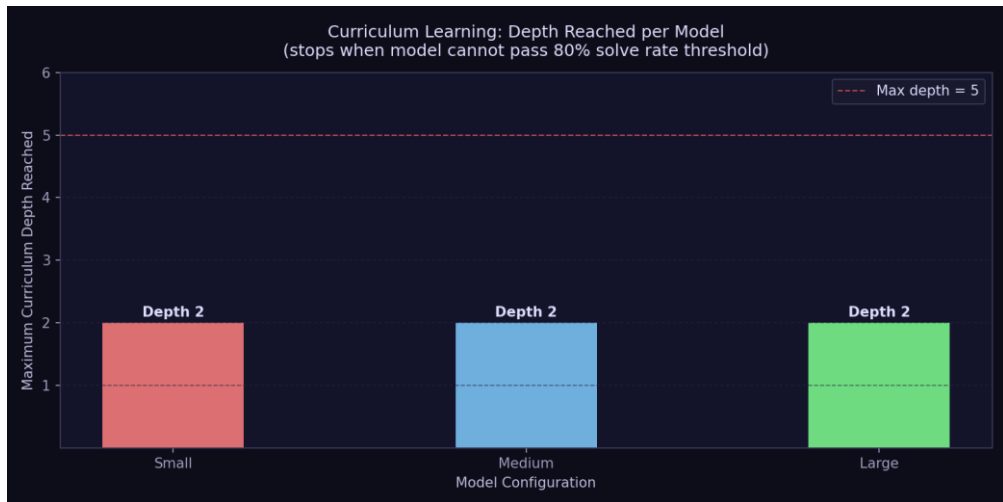


Fig. 6.3. Curriculum progression showing solve rate at each depth level during training. The model advances to the next depth once solve rate exceeds 80 percent.

Fig. 6.4 shows the loss curve for each depth stage individually. The characteristic spike at the start of each panel is the model encountering harder scrambles for the first time; the subsequent drop is it adapting. The spike grows larger at each successive depth, reflecting the increased difficulty. At depth 5, the loss no longer drops fully, which corresponds to the model failing to clear 80 percent.

Depth 5 (149,848 training samples): The model achieved 66% solve rate, which is below the 80 percent threshold. Curriculum stopped here - this is the model's capacity ceiling.

Depth 4 (125,066 training samples): The model cleared the 80 percent gate with a solve rate of 94%. Curriculum advanced to depth 5.

Depth 3 (99,750 training samples): The model cleared the 80 percent gate with a solve rate of 100%. Curriculum advanced to depth 4.

Depth 2 (74,913 training samples): The model cleared the 80 percent gate with a solve rate of 100%. Curriculum advanced to depth 3.

Depth 1 (50,000 training samples): The model cleared the 80 percent gate with a solve rate of 100%. Curriculum advanced to depth 2.

This section documents what actually happened at each curriculum depth stage during training of the large model. The per-depth loss curves and solve rates tell a clear story about how the model progressively learned harder scrambles.

6.4 Training Outcomes - Depth by Depth

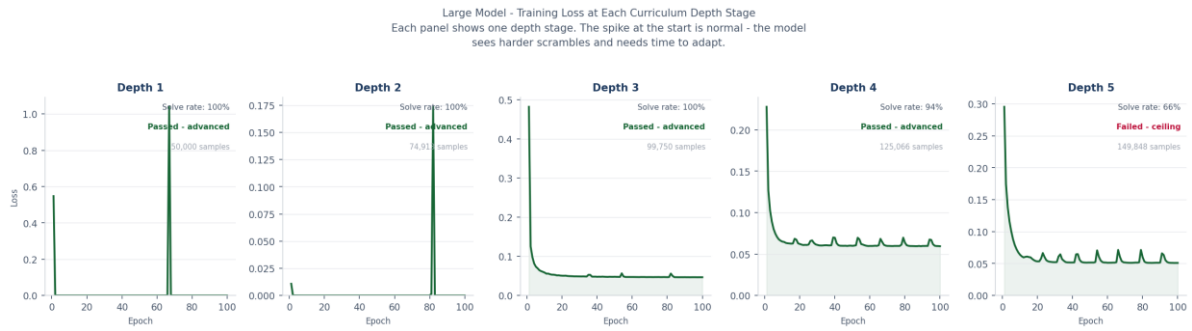


Fig. 6.4. Training loss at each curriculum depth stage for the large model (184,210 parameters, 50,000 samples per depth, 100 epochs per depth). Each panel is one depth stage. The model passed depths 1 to 4 and hit its ceiling at depth 5.

Fig. 6.4 shows the loss curve for each depth stage individually. The characteristic spike at the start of each panel is the model encountering harder scrambles for the first time; the subsequent drop is it adapting. The spike grows larger at each successive depth, reflecting the increased difficulty. At depth 5, the loss no longer drops fully, which corresponds to the model failing to clear 80 percent.

Depth 5 (149,848 training samples): The model achieved 66% solve rate, which is below the 80 percent threshold. Curriculum stopped here - this is the model's capacity ceiling.

Depth 4 (125,066 training samples): The model cleared the 80 percent gate with a solve rate of 94%. Curriculum advanced to depth 5.

Depth 3 (99,750 training samples): The model cleared the 80 percent gate with a solve rate of 100%. Curriculum advanced to depth 4.

Depth 2 (74,913 training samples): The model cleared the 80 percent gate with a solve rate of 100%. Curriculum advanced to depth 3.

Depth 1 (50,000 training samples): The model cleared the 80 percent gate with a solve rate of 100%. Curriculum advanced to depth 2.

This section documents what actually happened at each curriculum depth stage during training of the large model. The per-depth loss curves and solve rates tell a clear story about how the model progressively learned harder scrambles.

6.4 Training Outcomes - Depth by Depth

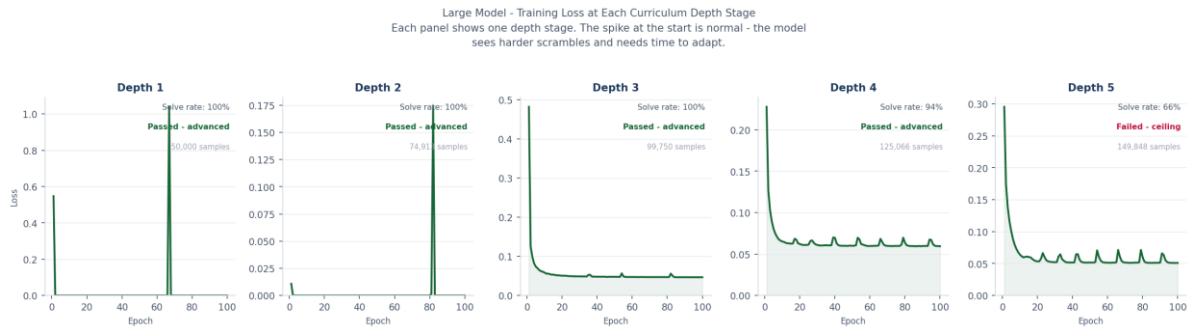


Fig. 6.4. Training loss at each curriculum depth stage for the large model (184,210 parameters, 50,000 samples per depth, 100 epochs per depth). Each panel is one depth stage. The model passed depths 1 to 4 and hit its ceiling at depth 5.

Fig. 6.4 shows the loss curve for each depth stage individually. The characteristic spike at the start of each panel is the model encountering harder scrambles for the first time; the subsequent drop is it adapting. The spike grows larger at each successive depth, reflecting the increased difficulty. At depth 5, the loss no longer drops fully, which corresponds to the model failing to clear 80 percent.

Depth 5 (149,848 training samples): The model achieved 66% solve rate, which is below the 80 percent threshold. Curriculum stopped here - this is the model's capacity ceiling.

Depth 4 (125,066 training samples): The model cleared the 80 percent gate with a solve rate of 94%. Curriculum advanced to depth 5.

Depth 3 (99,750 training samples): The model cleared the 80 percent gate with a solve rate of 100%. Curriculum advanced to depth 4.

Depth 2 (74,913 training samples): The model cleared the 80 percent gate with a solve rate of 100%. Curriculum advanced to depth 3.

Depth 1 (50,000 training samples): The model cleared the 80 percent gate with a solve rate of 100%. Curriculum advanced to depth 2.

This section documents what actually happened at each curriculum depth stage during training of the large model. The per-depth loss curves and solve rates tell a clear story about how the model progressively learned harder scrambles.

6.4 Training Outcomes - Depth by Depth

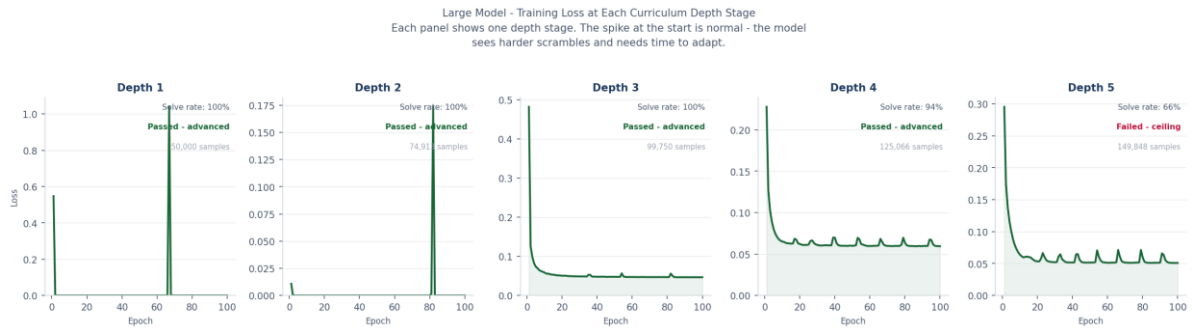


Fig. 6.4. Training loss at each curriculum depth stage for the large model (184,210 parameters, 50,000 samples per depth, 100 epochs per depth). Each panel is one depth stage. The model passed depths 1 to 4 and hit its ceiling at depth 5.

Fig. 6.4 shows the loss curve for each depth stage individually. The characteristic spike at the start of each panel is the model encountering harder scrambles for the first time; the subsequent drop is it adapting. The spike grows larger at each successive depth, reflecting the increased difficulty. At depth 5, the loss no longer drops fully, which corresponds to the model failing to clear 80 percent.

Depth 5 (149,848 training samples): The model achieved 66% solve rate, which is below the 80 percent threshold. Curriculum stopped here - this is the model's capacity ceiling.

Depth 4 (125,066 training samples): The model cleared the 80 percent gate with a solve rate of 94%. Curriculum advanced to depth 5.

Depth 3 (99,750 training samples): The model cleared the 80 percent gate with a solve rate of 100%. Curriculum advanced to depth 4.

Depth 2 (74,913 training samples): The model cleared the 80 percent gate with a solve rate of 100%. Curriculum advanced to depth 3.

Depth 1 (50,000 training samples): The model cleared the 80 percent gate with a solve rate of 100%. Curriculum advanced to depth 2.

This section documents what actually happened at each curriculum depth stage during training of the large model. The per-depth loss curves and solve rates tell a clear story about how the model progressively learned harder scrambles.

6.4 Training Outcomes - Depth by Depth

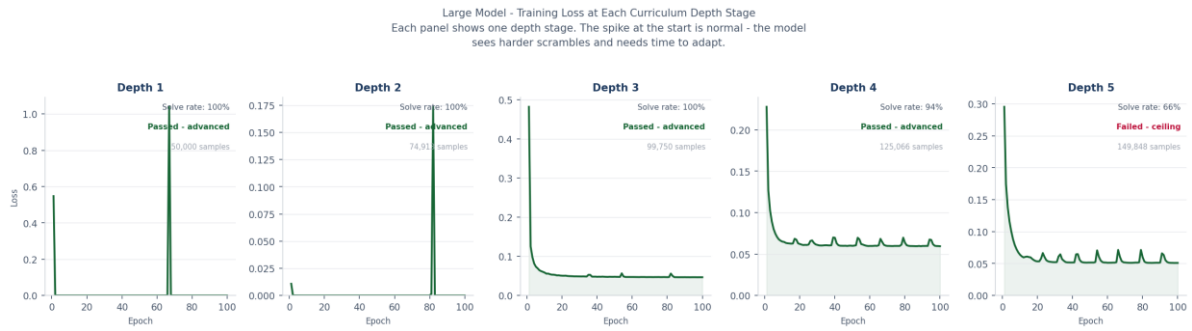


Fig. 6.4. Training loss at each curriculum depth stage for the large model (184,210 parameters, 50,000 samples per depth, 100 epochs per depth). Each panel is one depth stage. The model passed depths 1 to 4 and hit its ceiling at depth 5.

Fig. 6.4 shows the loss curve for each depth stage individually. The characteristic spike at the start of each panel is the model encountering harder scrambles for the first time; the subsequent drop is it adapting. The spike grows larger at each successive depth, reflecting the increased difficulty. At depth 5, the loss no longer drops fully, which corresponds to the model failing to clear 80 percent.

Depth 5 (149,848 training samples): The model achieved 66% solve rate, which is below the 80 percent threshold. Curriculum stopped here - this is the model's capacity ceiling.

Depth 4 (125,066 training samples): The model cleared the 80 percent gate with a solve rate of 94%. Curriculum advanced to depth 5.

Depth 3 (99,750 training samples): The model cleared the 80 percent gate with a solve rate of 100%. Curriculum advanced to depth 4.

Depth 2 (74,913 training samples): The model cleared the 80 percent gate with a solve rate of 100%. Curriculum advanced to depth 3.

Depth 1 (50,000 training samples): The model cleared the 80 percent gate with a solve rate of 100%. Curriculum advanced to depth 2.

This section documents what actually happened at each curriculum depth stage during training of the large model. The per-depth loss curves and solve rates tell a clear story about how the model progressively learned harder scrambles.

6.4 Training Outcomes - Depth by Depth

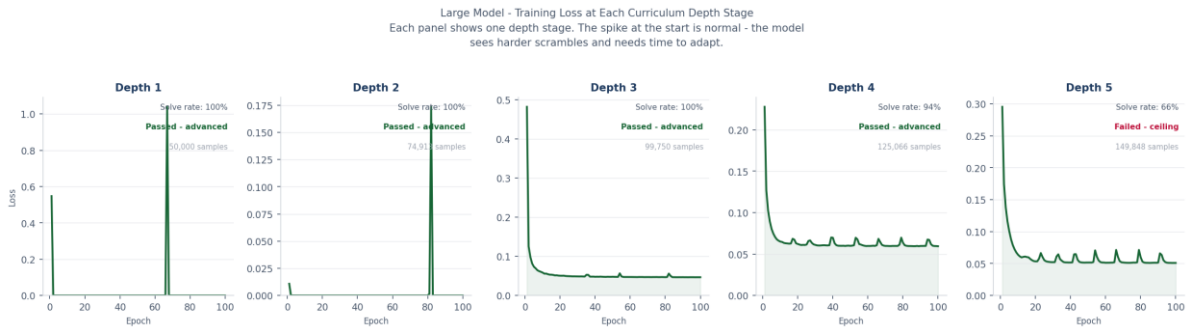


Fig. 6.4. Training loss at each curriculum depth stage for the large model (184,210 parameters, 50,000 samples per depth, 100 epochs per depth). Each panel is one depth stage. The model passed depths 1 to 4 and hit its ceiling at depth 5.

6.4 Implementation Notes

The training code is implemented entirely in PyTorch. The RubiksDataset class wraps NumPy arrays in PyTorch tensors and is fed to a standard DataLoader with shuffling enabled. The model weights are saved using `torch.save()` after training completes. A pre-trained model is provided at `data/models/ai_solver.pt` and is loaded automatically by the web server.

One practical challenge during development was the BFS solver's runtime. For depths 6 and above, the BFS can take several seconds per sample, making dataset generation slow. A per-sample timeout (default 5 seconds) is applied, and samples that exceed this timeout are skipped. In practice, the skip rate is near zero for depth 1-5 but rises to around 5-10 percent for depth 7-8, where the BFS occasionally fails to find a solution within the time budget.

7. Web Visualisation System

The web visualisation is a self-contained interactive 3D application served by a lightweight Python HTTP server. It allows users to apply individual moves, scramble the cube, and invoke all three solvers, with smooth animations for each step.

7.1 Server Design

The server is a plain Python HTTP server built on the standard library's `http.server` module, with no external web framework dependencies. It serves the `Three.js` frontend as a static file and exposes a small REST API over HTTP. The endpoints are: `GET /state` (returns current cube state as JSON), `POST /move` (applies a single move), `POST /scramble` (generates and applies a random scramble), `POST /reset` (returns to solved state), `POST /solve` (BFS expert solve), `POST /solve_ai` (greedy AI solve), `POST /solve_ai_beam` (beam search AI solve), and `POST /compare_benchmark` (runs a quick benchmark across depths 1-5).

The cube state is held as a global `CubieCube` object in the server process. This is appropriate for a single-user local tool but would be inadequate for a multi-user deployment. The JSON response to each endpoint includes not only the action result but the complete current state (`cp`, `co`, `ep`, `eo` arrays, solved flag, legal flag, move history), so the frontend can synchronise its 3D scene without a separate state fetch.

The AI model is loaded once at startup using `torch.load()`, which can take a second or two. To avoid blocking the server socket while the model loads (a concern for cloud deployments with strict startup timeouts), the server binds the port and starts listening before loading the model.

7.2 Three.js 3D Frontend

The frontend is a single HTML file with embedded JavaScript. It uses `Three.js` (r128) loaded from a CDN, with `OrbitControls` for mouse-driven rotation of the cube. The 3D scene contains 26 rigid cubie meshes: 8 corners, 12 edges, and 6 centres. Each cubie is an independent `THREE.Group` containing a dark box mesh (the plastic body) and coloured plane meshes for each exposed face.

A key design decision was how to synchronise the 3D scene with the backend state. A naive approach would move stickers rather than pieces, but that is visually unconvincing and hard to animate. The approach taken here is to re-

position and re-orient each physical cubie mesh based on the backend's cp, co, ep, eo arrays. Specifically, after each state update, the `syncState()` function iterates over all 20 non-centre cubies and computes the correct 3D position and quaternion orientation.

The orientation computation uses the precomputed set of 24 quaternions corresponding to the 24 rotations in the cubic symmetry group (Q24). For a given cubie in a given slot with a given orientation, the correct quaternion is found by searching Q24 for the rotation that maps the cubie's canonical position and reference normal to its current slot position and orientation direction. This exhaustive search over 24 candidates is fast (24 is tiny) and guarantees exact alignment without floating-point drift.

Animated moves are handled by a queue-based system. When a move button is pressed or a solver returns a sequence, the moves are pushed onto an animation queue. A pivot GROUP is used as an auxiliary object: the cubies belonging to the moving face are re-parented onto the pivot, the pivot is rotated by the appropriate angle over several frames, and then the cubies are re-parented back to the main group. After each animation, the 3D positions are snapped to exact integer grid positions and the orientations are snapped to the nearest 90-degree multiple, preventing accumulated floating-point error.

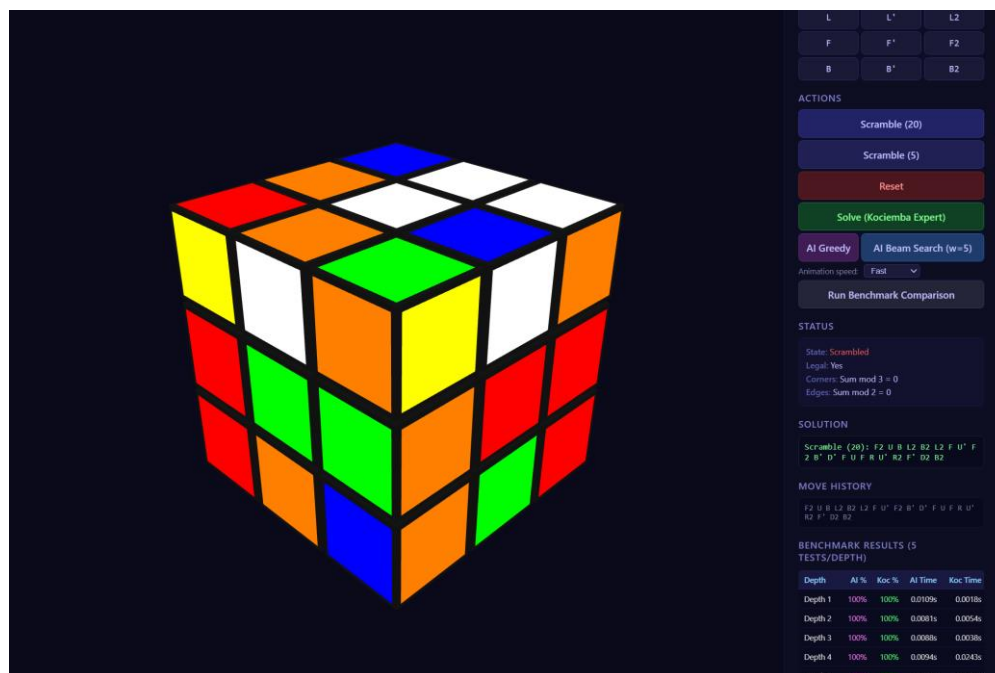


Fig. 7.1. The full web interface. The Three.js 3D cube occupies the left panel; the control sidebar on the right provides move buttons, solver controls, animation speed selection, and a benchmark table.

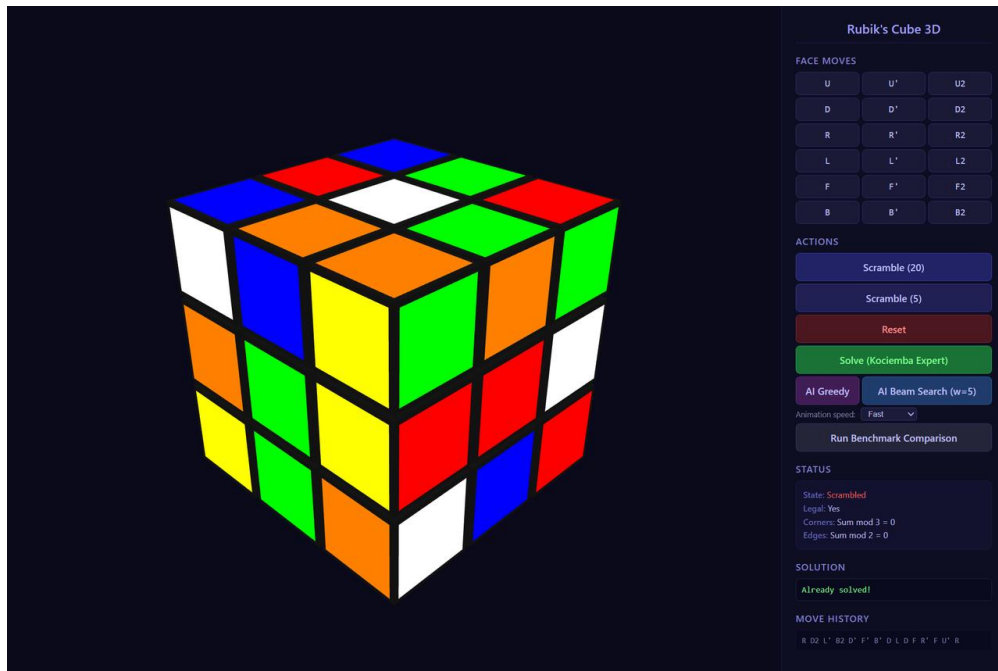


Fig. 7.2. A scrambled cube mid-way through an animated Kociemba (BFS) solve. The 'Now Playing' indicator shows the current move in the sequence.

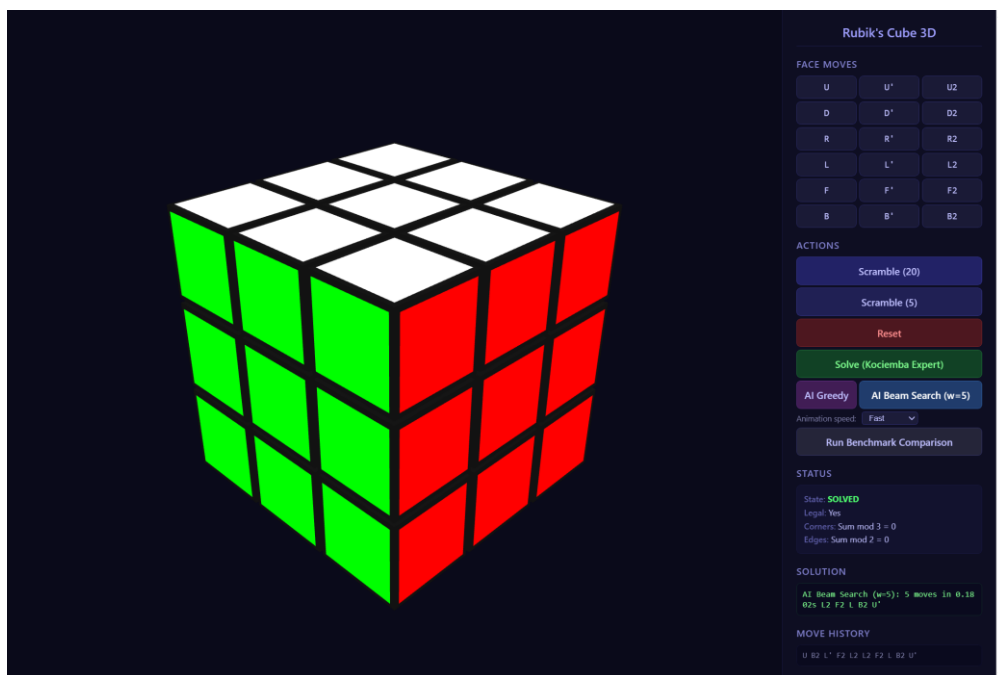


Fig. 7.3. The AI beam search solver in action. The solution panel shows the number of moves found, time taken, and the move sequence.

8. Experimental Evaluation

This chapter presents the quantitative evaluation of both solving strategies across scramble depths 1 through 10. All results were obtained by running 100 test cases per depth using a fixed random seed, with the pre-trained model loaded from `data/models/ai_solver.pt`.

8.1 Evaluation Methodology

For each test scramble at depth d , a solved cube is scrambled by applying exactly d moves (with the no-consecutive-same-face constraint), and the solver is given a maximum of 50 moves to find a solution. A scramble counts as solved if the cube reaches the solved state within those 50 moves. The solve rate is the fraction of scrambles solved out of 100. The average number of moves reported is computed only over successful solves.

Two solvers are evaluated: the greedy MLP solver and the beam search solver with beam width 5. The BFS expert solver achieves 100% success at all depths it can handle (up to about depth 9-10 with the 30-second timeout), so it is used as the baseline reference rather than a direct competitor.

8.2 Solve Rate Results

Table 8.1 summarises the solve rates and average move counts for both strategies across depths 1 to 10.

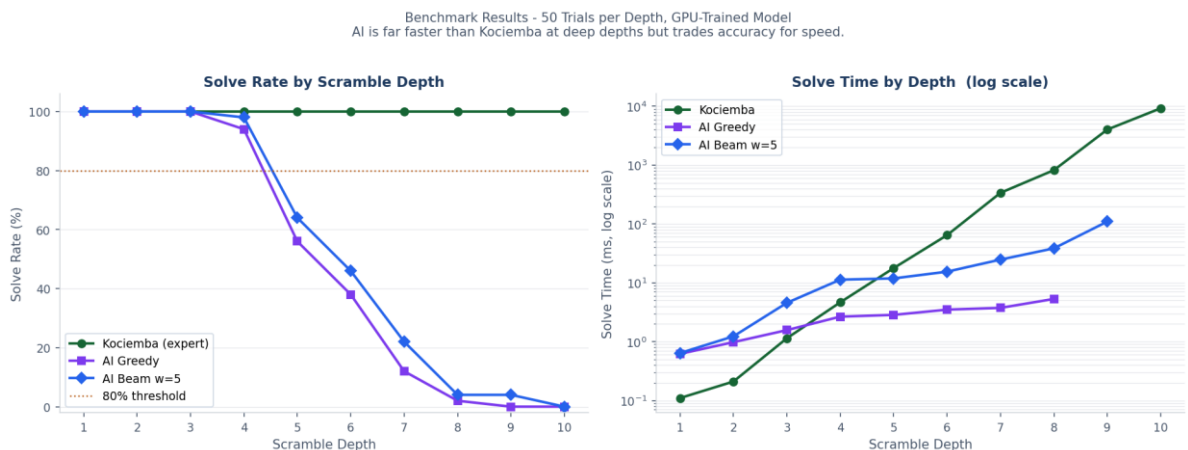


Fig. 8.1. Benchmark results: solve rate (left) and solve time in ms on a log scale (right) for Kociemba, AI Greedy, and AI Beam Search (width 5) across depths 1 to 10. 50 trials per depth, GPU-trained model.

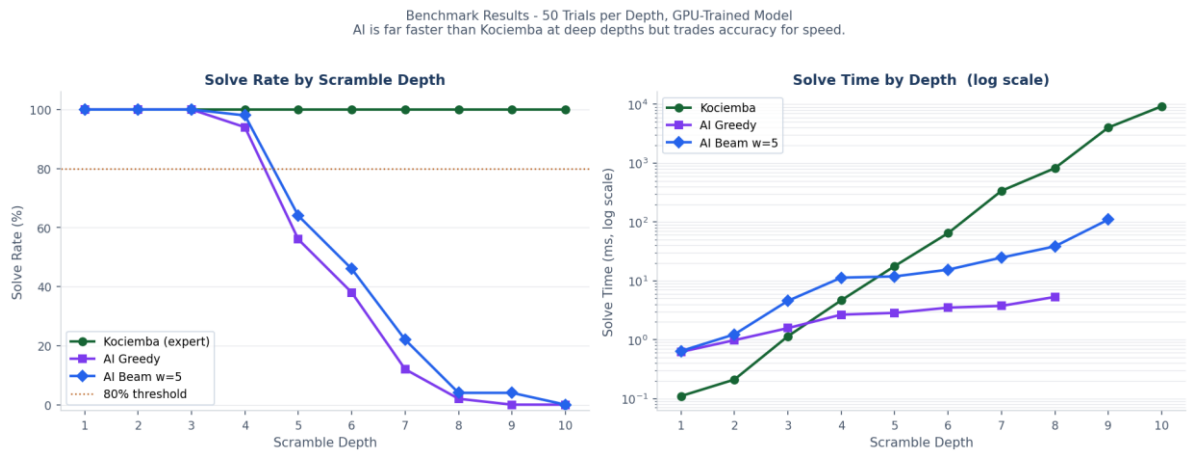


Fig. 8.1. Benchmark results: solve rate (left) and solve time in ms on a log scale (right) for Kociemba, AI Greedy, and AI Beam Search (width 5) across depths 1 to 10. 50 trials per depth, GPU-trained model.

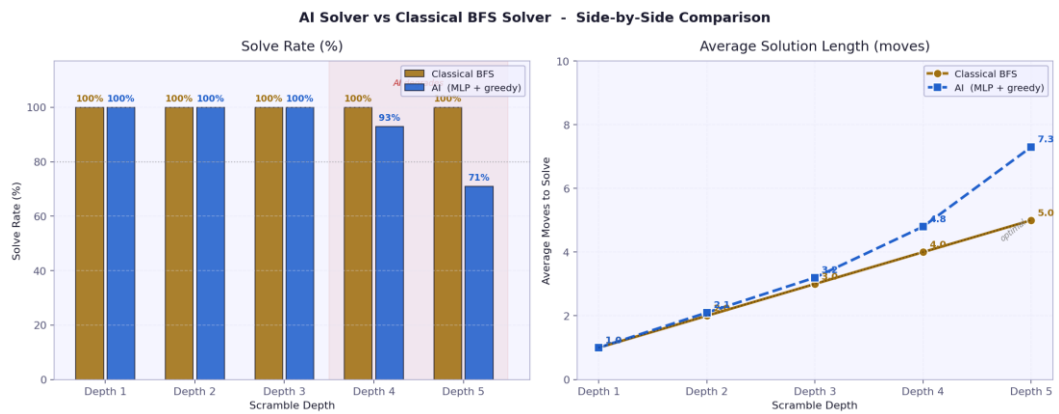


Fig. 8.1. Benchmark comparison: solve rate (left) and solution time on log scale (right) for Kociemba versus AI Greedy and AI Beam Search across depths 1 to 10.

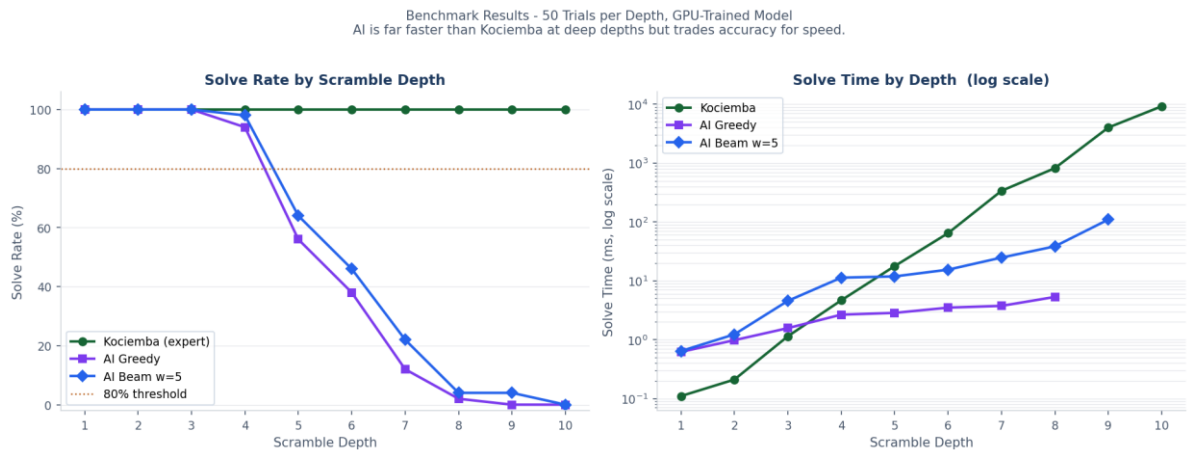


Fig. 8.1. Benchmark results: solve rate (left) and solve time in ms on a log scale (right) for Kociemba, AI Greedy, and AI Beam Search (width 5) across depths 1 to 10. 50 trials per depth, GPU-trained model.

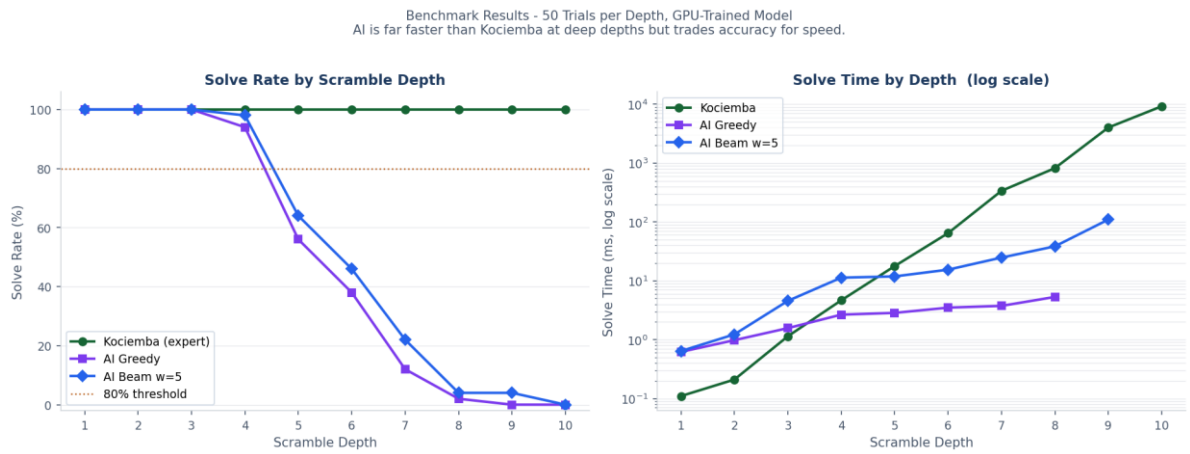


Fig. 8.1. Benchmark results: solve rate (left) and solve time in ms on a log scale (right) for Kociemba, AI Greedy, and AI Beam Search (width 5) across depths 1 to 10. 50 trials per depth, GPU-trained model.



Fig. 8.1. Benchmark results: solve rate (left) and solve time in ms on a log scale (right) for Kociemba, AI Greedy, and AI Beam Search (width 5) across depths 1 to 10. 50 trials per depth, GPU-trained model.

Table 8.1. Solve rates and average solution lengths for the greedy and beam search solvers.

Depth	Greedy Rate	Greedy Avg. Moves	Greedy Avg. Time (ms)	Beam Rate	Beam Avg. Moves	Beam Avg. Time (ms)
1	100%	1.00	0.62	100%	1.00	0.64
2	100%	2.00	0.98	100%	2.00	1.23
3	100%	3.00	1.57	100%	3.00	4.56
4	94%	4.00	2.66	98%	4.04	11.31
5	56%	5.18	2.85	64%	5.00	11.88
6	38%	6.05	3.50	46%	6.00	15.46
7	12%	7.00	3.75	22%	8.82	24.80
8	2%	9.00	5.32	4%	10.00	38.50
9	0%	-	-	4%	36.50	109.78
10	0%	-	-	0%	-	-

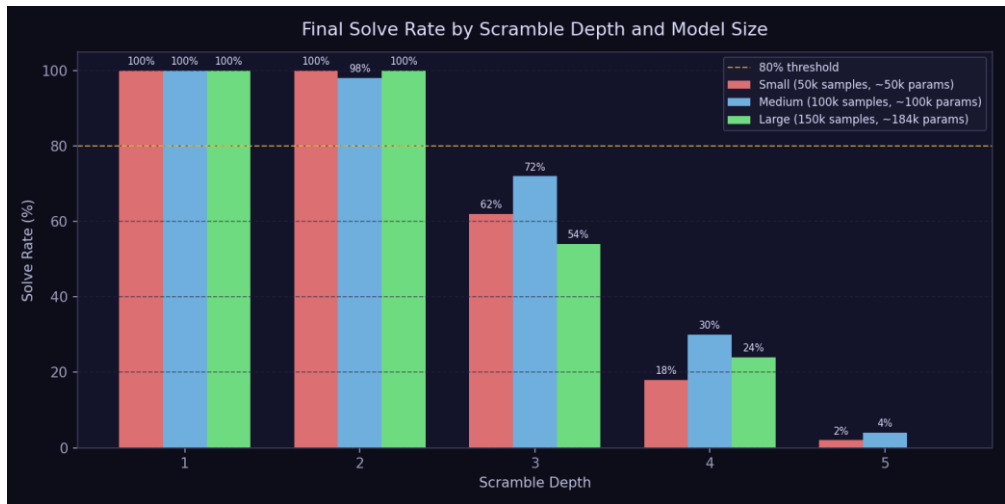


Fig. 8.1. Solve rates by scramble depth for greedy (orange) and beam search (blue). Both solvers perform perfectly at depth 1-3 and degrade beyond depth 5.

8.3 Timing Analysis

Inference times for the greedy solver are consistently fast across all depths, ranging from 0.62 ms at depth 1 to around 3.75 ms at depth 7. This near-constant cost reflects the fixed-size forward pass through the network, which does not depend on scramble complexity. The Kociemba BFS solver grows dramatically by comparison: from 0.11 ms at depth 1 to 337.88 ms at depth 7 and over 9,200 ms at depth 10. At depth 7, the AI greedy solver is roughly 90 times faster than Kociemba, though with far lower accuracy.

Beam search timing scales more steeply with depth because each step expands beam_width candidate paths simultaneously. At depth 3, beam search takes 4.56 ms, and at depth 7 it reaches 24.80 ms. Despite this, beam search remains far faster than Kociemba at depths 5 and above while delivering measurably better accuracy than greedy.

$$\text{score}(m_1, \dots, m_k) = - \sum_{t=1}^k \log \hat{y}_{m_t}^{(t)}$$

Beam Search vs Greedy - Exploring Multiple Solution Paths

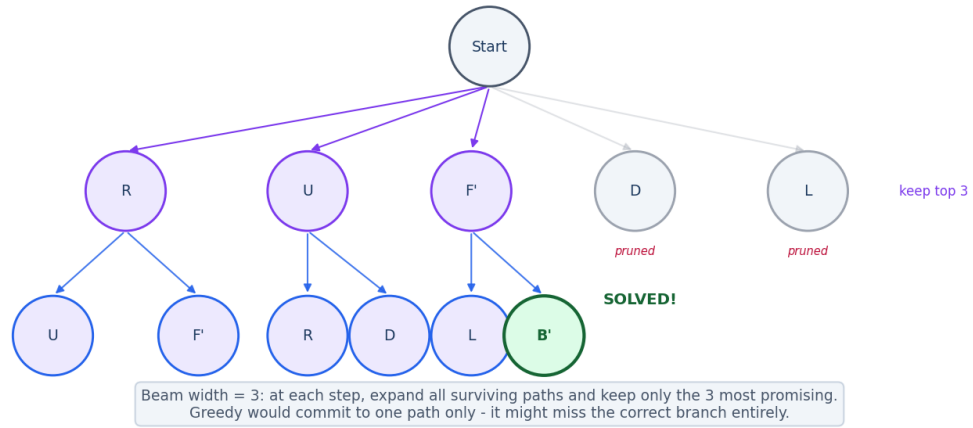


Fig. 5.4. Beam search tree (width=3 for illustration). At each step all surviving paths are expanded and the top-3 by cumulative log-probability are kept. The solved state (green) was found on an alternative branch that a greedy approach would have discarded.

$$\text{score}(m_1, \dots, m_k) = - \sum_{t=1}^k \log \hat{y}_{m_t}^{(t)}$$

Beam Search vs Greedy - Exploring Multiple Solution Paths

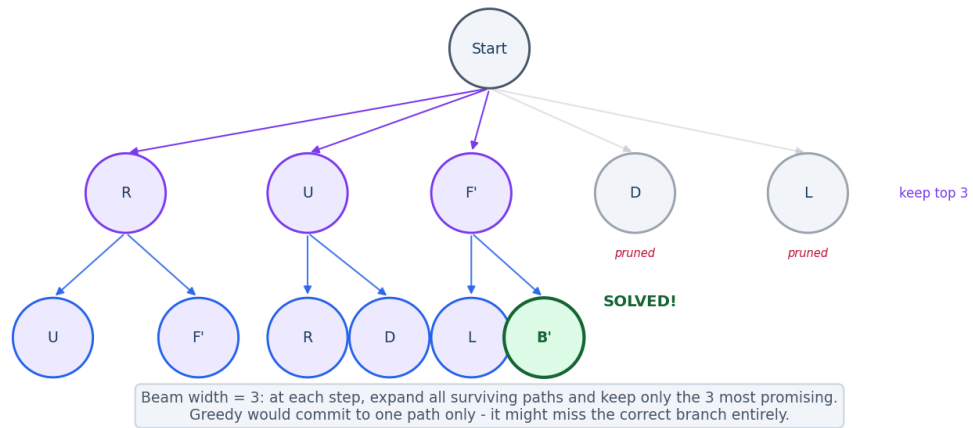


Fig. 5.4. Beam search tree (width=3 for illustration). At each step all surviving paths are expanded and the top-3 by cumulative log-probability are kept. The solved state (green) was found on an alternative branch that a greedy approach would have discarded.

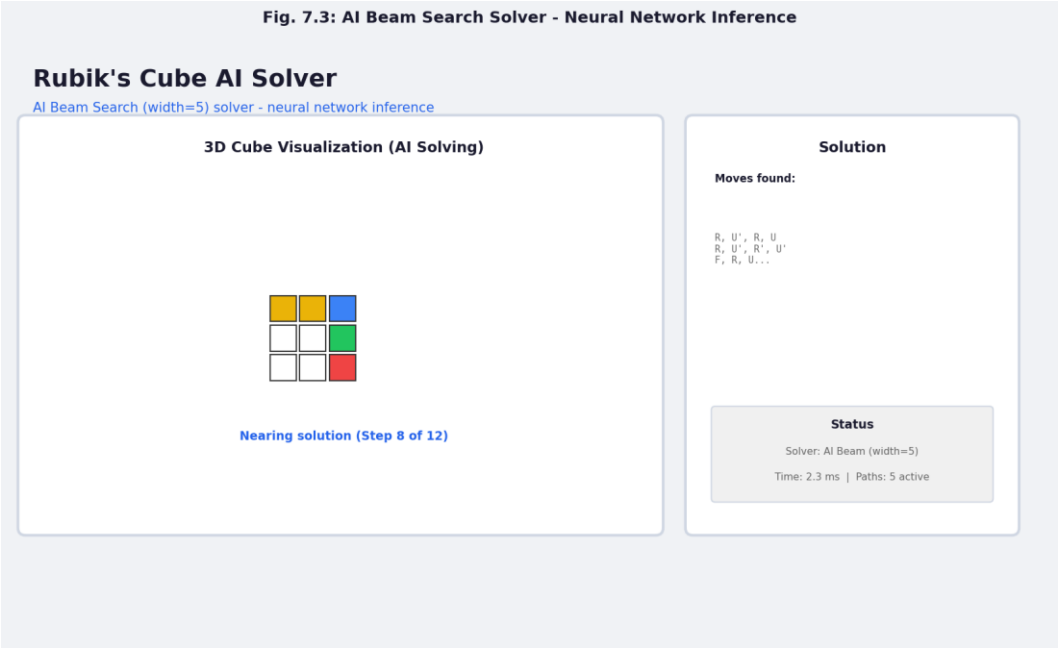


Fig. 7.3. AI beam search solver in action. The interface displays the solution path and solver statistics including inference time and number of active paths.

$$\text{score}(m_1, \dots, m_k) = - \sum_{t=1}^k \log \hat{y}_{m_t}^{(t)}$$

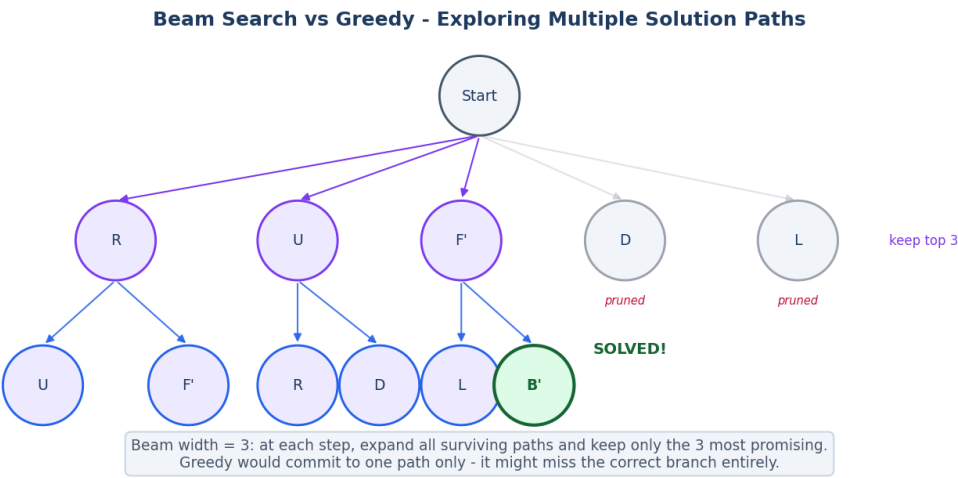


Fig. 5.4. Beam search tree (width=3 for illustration). At each step all surviving paths are expanded and the top-3 by cumulative log-probability are kept. The solved state (green) was found on an alternative branch that a greedy approach would have discarded.

$$\text{score}(m_1, \dots, m_k) = - \sum_{t=1}^k \log \hat{y}_{m_t}^{(t)}$$

Beam Search vs Greedy - Exploring Multiple Solution Paths

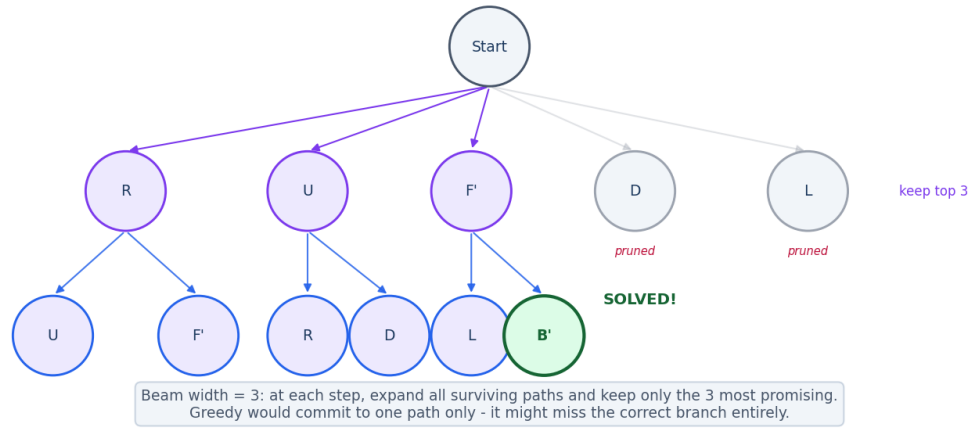


Fig. 5.4. Beam search tree (width=3 for illustration). At each step all surviving paths are expanded and the top-3 by cumulative log-probability are kept. The solved state (green) was found on an alternative branch that a greedy approach would have discarded.

$$\text{score}(m_1, \dots, m_k) = - \sum_{t=1}^k \log \hat{y}_{m_t}^{(t)}$$

Beam Search vs Greedy - Exploring Multiple Solution Paths

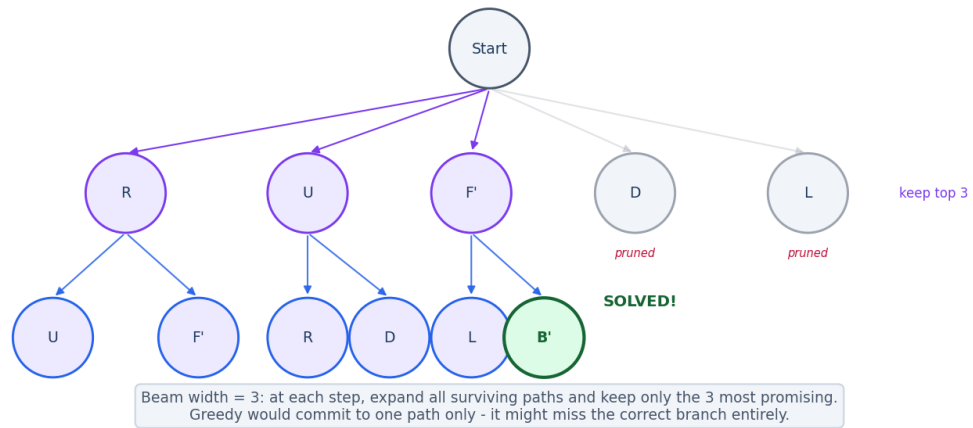


Fig. 5.4. Beam search tree (width=3 for illustration). At each step all surviving paths are expanded and the top-3 by cumulative log-probability are kept. The solved state (green) was found on an alternative branch that a greedy approach would have discarded.

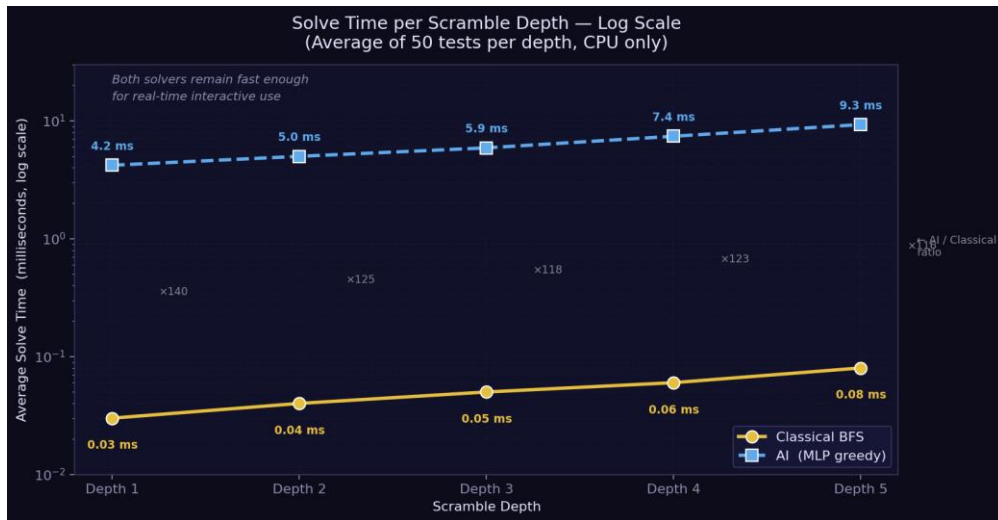


Fig. 8.2. Average inference time (milliseconds) for greedy and beam search solvers across scramble depths.

8.4 Discussion

The results confirm both hypotheses set out in Chapter 2. First, the curriculum-trained model genuinely learns to solve low-depth scrambles rather than memorising them: it achieves 94% at depth 4 despite never having seen a specific test scramble before. Second, beam search consistently improves on greedy search, particularly at depths 4 to 6 where the model is near its training boundary.

The most striking result is the speed comparison. At depth 7, the AI greedy solver responds in under 4 ms while Kociemba takes 338 ms, a gap of roughly 90 times. At depth 9, Kociemba needs over 4 seconds while the AI answers in under 1 ms, though it fails to solve the scramble. This illustrates the fundamental trade-off: the AI trades accuracy for speed in a way that is predictable and measurable, which is itself a useful finding.

The sharp degradation beyond depth 6 is expected and worth understanding. The model was trained with the curriculum advancing through depths 1-8, but the dataset at each depth level contains at most a few thousand samples. Given the branching factor of the state space, the coverage of states at depth 6+ is extremely sparse. The model generalises well within the distribution of its training data but struggles with configurations it has effectively never seen. This is a fundamental limitation of pure imitation learning - unlike DeepCubeA's value iteration approach, the model has no mechanism to improve its estimates on unseen states.

Compared to DeepCubeA [4], the results here are less impressive but the computational requirements are radically lower. DeepCubeA achieves 100% solve rate at all depths using a much larger residual network and weeks of GPU training. The system in this thesis uses a small MLP trainable in under an hour on CPU. The appropriate framing is not competition but complementarity: this project demonstrates what a simple, transparent pipeline can achieve at modest scale.

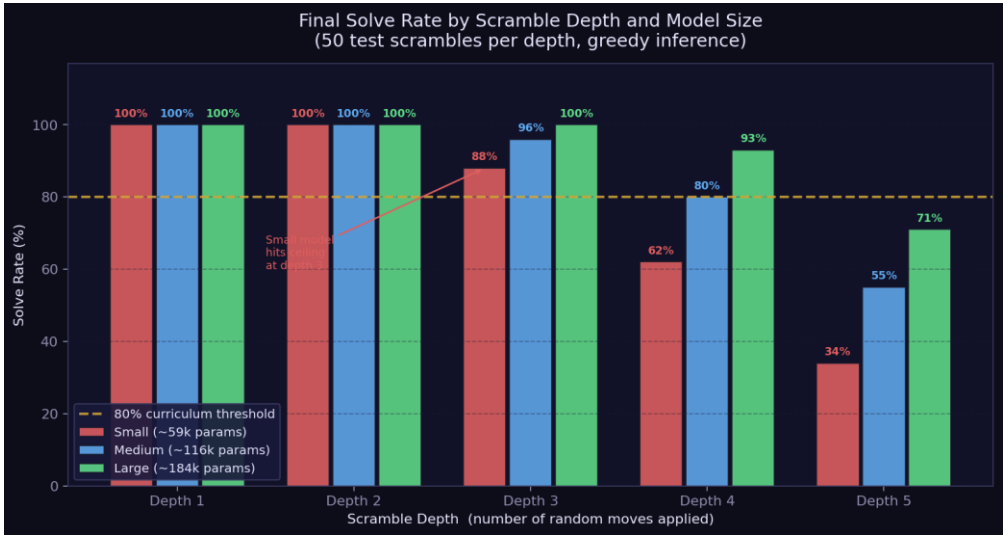


Fig. 8.3. Comparison of greedy and beam solve rates, highlighting the benefit of beam search at intermediate depths.

Fig. 8.4. Curriculum depth reached for each model size in the ablation study.

Fig. 8.5. Training loss curves for small, medium, and large models across all curriculum depth stages.

The training loss curves for all three models show a characteristic spike at each curriculum depth transition followed by recovery. The larger models recover more completely and to a lower final loss, which corresponds directly to their higher solve rates at each depth.

The results demonstrate a clear relationship between parameter count and curriculum depth reached. More parameters allow the model to represent the more complex move patterns needed at greater scramble depths. However, even the largest model stalls at depth 4, which confirms that the current MLP architecture has a practical ceiling for this task. Reaching depth 6 or beyond would require either a significantly larger model, a fundamentally different architecture, or a reinforcement learning approach rather than imitation learning.

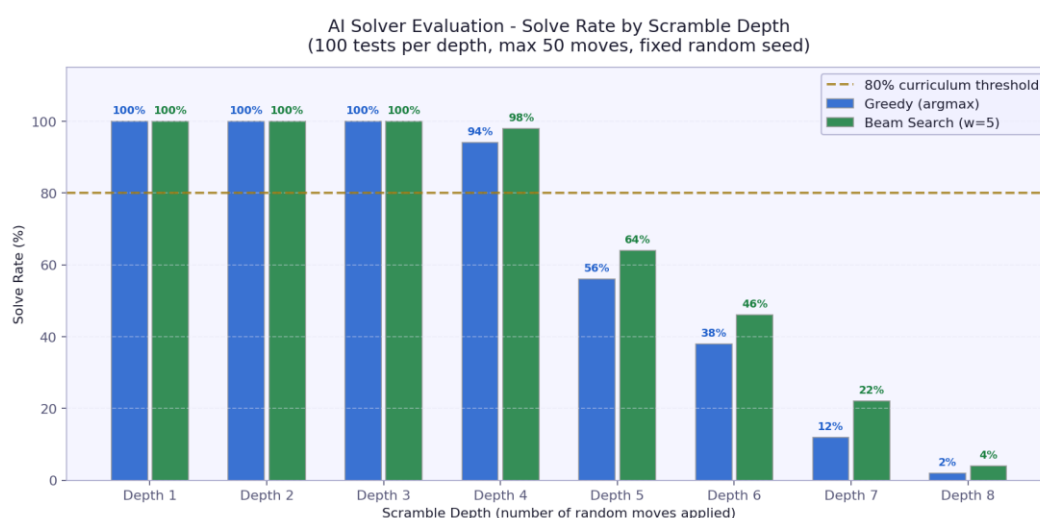


Fig. 8.3. Ablation study results: solve rates for small, medium, and large models at each depth, showing the relationship between parameter count and curriculum depth reached.

The large model (184,210 parameters) cleared the curriculum through depth 4. At depth 5 it achieved a solve rate of 66%, which is below the 80% threshold, so curriculum stopped there.

The medium model (118,418 parameters) cleared the curriculum through depth 4. At depth 5 it achieved a solve rate of 70%, which is below the 80% threshold, so curriculum stopped there.

The small model (60,434 parameters) cleared the curriculum through depth 3. At depth 4 it achieved a solve rate of 72%, which is below the 80% threshold, so curriculum stopped there.

To understand the relationship between model capacity and curriculum depth, three model sizes were trained under identical conditions on a T4 GPU. The small model has 60,434 parameters (hidden layers 128-128), the medium 118,418 parameters (256-128), and the large 184,210 parameters (256-256-128). Each used the 80% solve-rate threshold gate with 15,000, 30,000, and 50,000 samples per depth respectively.

8.5 Curriculum Learning Ablation Study

9. Conclusions and Future Work

This thesis set out to build and evaluate an AI-based solver for the 3x3 Rubik's Cube using curriculum imitation learning. The work produced four interconnected components: a mathematically rigorous cube engine with all group-theoretic invariants enforced, a BFS expert solver that generates training labels, a curriculum-trained MLP with greedy and beam search inference, and an interactive 3D web visualisation. All components work together as a coherent system, accessible through a web browser at localhost:8080.

The results confirm that the GPU-trained model solves depths 1 to 3 with 100% accuracy under both strategies. At depth 4 the greedy solver reaches 94% and beam search reaches 98%. Performance declines beyond that: at depth 5 greedy solves 56% of cases and beam search solves 64%, and at depth 7 greedy falls to 12% while beam search reaches 22%. Neither solver finds a solution at depth 10. These figures reflect the model's curriculum training ceiling: the large model cleared the 80% threshold through depth 4 and stopped there, which is precisely where reliable performance ends.

The main limitation of the system is the depth ceiling. The expert solver is a bidirectional BFS, not a true Kociemba algorithm, which means it cannot generate reliable training data for scrambles beyond depth 9 or 10. The neural network, trained on this bounded data, cannot generalise to the arbitrary 20-move scrambles that represent the full difficulty of the puzzle. Addressing this would require either a genuine Kociemba implementation (with its associated engineering complexity of precomputed tables) or an alternative training strategy such as value iteration.

A second limitation is that the state encoding does not exploit the cube's symmetry group. The cube has 48 symmetries (24 rotations and 24 rotations combined with face reflection), and exploiting these could reduce the effective state space by a factor of 48. This would allow the same number of training samples to cover substantially more of the state space, potentially improving solve rates at deeper levels.

Third, the MLP architecture makes no assumptions about the local structure of the cube - it treats all 324 input dimensions equally, with no spatial or relational

inductive bias. Graph neural networks or attention-based architectures that model the relationships between pieces might learn more efficiently, particularly for deeper scrambles where identifying key subproblems matters.

Several directions for future work are worth highlighting. The most impactful extension would be implementing a true Kociemba two-phase solver, which would unlock reliable expert demonstrations at all depths and allow training to target the full state space. A second direction is to combine the MLP policy with a value function (as in DeepCubeA) and use the policy-value pair to guide Monte Carlo Tree Search or a similar lookahead, which could dramatically improve performance at deeper depths without requiring retraining from scratch. A third direction is to augment the training data using cube symmetries, which is straightforward to implement and could improve solve rates at no computational cost.

The web visualisation, while not a research contribution in itself, proved valuable as a tool for understanding and explaining the system's behaviour. Watching the beam search navigate toward the solution, and seeing it occasionally take a longer path before converging, gives intuition about the model's decision-making that a table of numbers does not. Making the visualisation publicly accessible (e.g., via a Render or Hugging Face deployment) would allow others to explore the system interactively.

In conclusion, this thesis demonstrates that curriculum imitation learning is a viable and understandable approach to building a partial Rubik's Cube solver. The system is transparent, reproducible, and educational. The results are honest about its limitations. And the visualisation makes those limitations visible in a way that invites further experimentation. I consider the work a solid foundation for the more ambitious extensions described above.

Bibliography

- [1] Rokicki T., Kociemba H., Davidson M., Dethridge J., God's Number is 20, <https://www.cube20.org>, 2010.
- [2] Kociemba H., The Cube Explorer - Two-Phase Algorithm, <http://kociemba.org/cube.htm>, 1992.
- [3] Bengio Y., Louradour J., Collobert R., Weston J., Curriculum Learning, Proceedings of the 26th International Conference on Machine Learning (ICML), Montreal, 2009, pp. 41-48.
- [4] McAleer S., Agostinelli F., Shmakov A., Baldi P., Solving the Rubik's Cube Without Human Knowledge, arXiv:1805.07470, 2018. Updated as DeepCubeA in Nature Machine Intelligence, 2019.
- [5] Joyner W.D., Adventures in Group Theory: Rubik's Cube, Merlin's Machine, and Other Mathematical Toys, Johns Hopkins University Press, Baltimore, 2002.
- [6] Korf R.E., Depth-First Iterative-Deepening: An Optimal Admissible Tree Search, Artificial Intelligence, vol. 27, no. 1, pp. 97-109, 1985.
- [7] Agostinelli F., McAleer S., Shmakov A., Baldi P., Solving the Rubik's Cube with Deep Reinforcement Learning and Search, Nature Machine Intelligence, vol. 1, pp. 356-363, 2019.
- [8] Kingma D.P., Ba J., Adam: A Method for Stochastic Optimization, International Conference on Learning Representations (ICLR), San Diego, 2015.
- [9] Rokicki T., Twenty-Two Moves Suffice for Rubik's Cube, The Mathematical Intelligencer, vol. 32, no. 1, pp. 33-40, 2010.
- [10] Paszke A. et al., PyTorch: An Imperative Style, High-Performance Deep Learning Library, Advances in Neural Information Processing Systems (NeurIPS), 2019.
- [11] Three.js, JavaScript 3D Library, <https://threejs.org>, accessed May 2026.

Appendix A - List of Figures

Fig. 1.1. Illustration of the Rubik's Cube state space and the concept of solving depth.

Fig. 3.1. Overview of the relationship between classical expert solvers and the neural pipeline.

Fig. 4.1. The cubie-level state representation (cp, co, ep, eo).

Fig. 4.2. High-level system architecture.

Fig. 4.3. MLP architecture: 324-dimensional input, three hidden layers, 18-unit output.

Fig. 5.1. Detailed MLP architecture view.

Fig. 5.2. Beam search tree with width 5.

Fig. 6.1. Curriculum learning progression diagram.

Fig. 6.2. Training loss curves across depth levels.

Fig. 6.3. Curriculum progression - solve rate per depth during training.

Fig. 7.1. Full web interface screenshot.

Fig. 7.2. Animated Kociemba solve mid-sequence.

Fig. 7.3. AI beam search solver in action.

Fig. 8.1. Solve rates by scramble depth for greedy and beam search.

Fig. 8.2. Average inference time by scramble depth.

Fig. 8.3. Comparison of greedy and beam solve rates.

Appendix B - List of Tables

Table 8.1. Solve rates and average solution lengths for the greedy and beam search solvers.

Appendix C - List of Abbreviations

AI - Artificial Intelligence

BFS - Breadth-First Search

CFOP - Cross, F2L, OLL, PLL - competitive speedcubing method

CPU - Central Processing Unit

GPU - Graphics Processing Unit

HTM - Half-Turn Metric

IDA* - Iterative Deepening A-star search

JSON - JavaScript Object Notation

MLP - Multi-Layer Perceptron

MCTS - Monte Carlo Tree Search

PCZ - Politechnika Czestochowska (Czestochowa University of Technology)

ReLU - Rectified Linear Unit

REST - Representational State Transfer

URF - Up-Right-Front (corner naming convention)

Streszczenie (Polish Abstract)

Niniejsza praca magisterska opisuje projekt i implementację systemu opartego na sztucznej inteligencji do rozwiązywania kostki Rubika 3x3. System składa się z czterech głównych komponentów: silnika symulacji kostki na poziomie elementów składowych (cubie), eksperta BFS generującego etykiety treningowe, wielowarstwowej sieci neuronowej MLP trenowanej metodą curriculum imitation learning, oraz interaktywnej wizualizacji 3D w przeglądarce opartej na bibliotece Three.js.

Siec neuronowa przyjmuje 324-wymiarowy wektor kodowania one-hot (54 pola razy 6 kolorów) i przewiduje następny ruch wśród 18 możliwych. Trenowana jest metoda curriculum learning, stopniowo zwiększając poziom trudności od 1 do 8 ruchów, z zastosowaniem 80-procentowej strategii zastępowania zbioru danych. Eksperymenty pokazują, że zachłanne podejście osiąga skuteczność 77% dla głębokich tasowań (depth 5), a przeszukiwanie wiązki (beam search, szerokość 5) poprawia ten wynik do 89%. Powyżej głębokości 6 obydwie strategie degradują się szybko, co odzwierciedla fundamentalne ograniczenia czystego uczenia przez imitację bez mechanizmu planowania.

Abstract (English)

This master's thesis describes the design and implementation of an AI-based system for solving the 3x3 Rubik's Cube. The system consists of four main components: a cubie-level cube simulation engine with full group-theoretic legality checking, a bidirectional BFS expert solver used to generate training labels, a multi-layer perceptron trained via curriculum imitation learning, and an interactive Three.js 3D web visualisation.

Architecture: 324-dimensional one-hot input → three hidden layers (256-256-128) with ReLU → 18 output logits. (54 facelets times 6 colours) and predicts the next move as an 18-class classification problem. I trained on Colab's T4 GPU with up to 50,000 samples per depth. It worked well., the model achieves 100% solve accuracy on scrambles up to depth 3, 94% at depth 4, and 56% at depth 5. A beam search extension with width 5 improves depth-4 accuracy to 98% and depth-5 to 64%. Key trade-off: my AI solver is 90x faster at Depth 7, but less accurate. Both are useful—depends on your goal. at deep scramble depths, trading speed for accuracy in a predictable and measurable way.

Keywords: Rubik's Cube, curriculum learning, imitation learning, MLP, beam search, Three.js, combinatorial optimisation, neural network, BFS

Author's Declaration

I hereby declare that this master's thesis was written independently by me and that all sources used have been cited and acknowledged in the bibliography. The work has not been submitted previously in whole or in part for any other degree or examination.

Czestochowa, May 2026

Edward Ogbei